openvaf-r — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

July 2026

Contents

openvaf-r — Full Change Report

Every modification applied to the OpenVAF-Reloaded compiler in this project, with its reason. The baseline is the pristine OpenVAF-Reloaded source tree (as vendored in the project's `original/` snapshot); the current state is the tree committed in this repository under `OpenVAF-master-20260610/`. Each entry links the enhancement write-up in `enhancements_doc/` that carries the full engineering detail and the verifying example suite. The companion document ngspice_changes_full-report.md covers the simulator side.

> Maintenance note: this report is updated whenever an enhancement touches compiler sources. The per-enhancement index at the end tells you the last change it covers.

Scope summary. ~120 modified source files across the compiler pipeline, two new files (`hir/src/elaborate.rs` — the elaboration pass — and `hir_def/src/item_tree/diagnostics.rs`), plus three classes handled wholesale: the `test_data/` fixtures and snapshots (95 files — new test inputs and snapshots regenerated whenever a feature legitimately changed descriptor contents), the `integration_tests/*/*.osdi` reference objects (regenerated at each ABI bump), and the removal of the AGPL `va-cask` submodule. The compiler builds warning-free; every change below was regression-gated by the example-suite arsenal, the integration suite, and the VA_TEST industry-model corpus.

---

1. Lexer — **`openvaf/lexer/`** (**`lib.rs`**, **`cursor.rs`**)

- Based integer literals (`8'hFF`, `'sb101`): the lexer had only a commented sketch; underscores crashed it. Both the sized and unsized forms now lex, requiring a digit after the base (a silent-0 trap otherwise) (E-46).
- Don't-care digits `x/X/z/Z/?` accepted in the binary/octal/hex digit runs for `casex`/`casez` items (E-78).
- Escaped identifiers (`\my-net!`) lexed per the LRM (E-46).
- **`// comment at EOF`** hang fixed: with no trailing newline the line-comment loop spun forever on the EOF character — the only EOF-unsafe loop in the lexer (E-35).

2. Tokens and grammar — **`openvaf/tokens/`**, **`openvaf/parser/`**, **`veriloga.ungram`**, **`sourcegen/`**

`tokens/src/parser/generated.rs` gains the keywords and node kinds each language feature needed, threaded through `parser/src/grammar/*` (`stmts.rs`, `expressions.rs`, `items.rs`, `items/module.rs`), the grammar source of truth `syntax/veriloga.ungram`, and the `sourcegen/` generators (`ast/src.rs`, `hir_builtins.rs`, `mir_instructions.rs` — the templates the generated token/AST/builtin/instruction tables come from):

- `do … while` post-test loops (E-19);
- `paramset`/`endparamset` blocks (E-21);
- `{…}` concatenation and `{n{…}}` replication as real operators, with `'{…}` remaining the typed aggregate (E-34);
- array-literal expressions actually parsed (they were fully scaffolded but unreachable) (E-4, E-14);
- vectored (bus) net declarations and bit-selects (E-3), multi-index bit-selects for N-D arrays (E-15), and the LRM name-then-range declaration order (E-18);
- module instantiation items (E-5), `generate for`/`genvar` (E-8), and `generate if`/`generate case` with optional block labels (previously mis-parsed into a broken `GENERATE_FOR`; E-67);
- `defparam` — a reserved word with no grammar rule (E-58);
- event OR lists `@(cross(…) or timer(…))` via a new `or` keyword and looped event grammar (E-59);
- `casex`/`casez` sharing the case rule, the keyword token carrying the flavor (E-78);

- `@(initial_step)`/`@(final_step)` with analysis-phase lists (E-7, E-53);
- indirect branch assignment `V(x): V(y) == expr;` (E-2);
- the `<<<`/`>>>` arithmetic-shift tokens (plus a pre-existing lexer bug fix) (E-6);
- hierarchical parameter override targets (`#(.blk.p(4))`): the extra `.segments` are consumed so the parse stays clean instead of cascading -- elaboration then rejects the block-scoped override with a targeted diagnostic (E-87);
- hierarchical branch reference tails (`.branch(a,b)` / `.branch(<p>)`) swallowed into the path node so the enclosing item's CST stays whole for the elaboration rewrite (a keyword in path position used to shred the item, leaving the hole scanner truncated text) (E-86);
- part-selects in instance connections (`inst (out[3:2], in)`): the bit-select bracket accepts an optional `: expr`, with the colon token in the CST distinguishing a range from multi-dimensional indexing — no new node kind (E-85);
- module-level **`generate for`**/**`if`**/**`case`** without the optional **`generate`**/**`endgenerate`** keywords: `module_items` handled a `generate … endgenerate` region but had no bare `FOR_KW/IF_KW/CASE_KW` arm, so a keyword-less loop at module scope fell to error recovery — `unexpected token 'for'`, or a silent drop of the loop when a following `analog` block let recovery resync. New arms parse it into the same `GENERATE_FOR/IF/CASE` nodes the nested/`generate`-wrapped forms produce (E-96);
- panic-free direction parsing: `port_decl`/`func_arg` asserted the next token was a direction (`bump_ts`), which any non-Verilog text reaching a module-head port list turned into a compiler crash; both now emit a diagnostic with one token of forced progress (E-84);
- operator precedence corrected against LRM Table 4-2: `%` bound tighter than `*`/`/` (so `6*7%4` evaluated to 18, not 2), and xnor split from xor (E-38);
- derived-nature parents emitted the wrong node kind (`NAME_REF` where the AST wanted `Path`), leaving the entire inheritance machinery unreachable (E-39).

3. Preprocessor — **`openvaf/preprocessor/`** (+ **`basedb`** diagnostics)

- Ten compiler directives that previously hard-failed as undefined macros: `` `default_discipline ``, `` `celldefine ``/`` `endcelldefine ``, `` `unconnected_drive ``/`` `nounconnected_drive ``, `` `timescale ``, `` `line ``, `` `pragma ``, `` `undefineall ``, `` `default_nettype `` (E-6).
- **`` `default_transition ``** parsed and threaded to `transition()` lowering (E-47).
- Recursive macro expansion crashed the compiler (stack overflow, direct and mutual): the `MacroRecursion` diagnostic existed but was never emitted, and its report renderer was a `todo!()`. The guard pushes around the macro *body* expansion only — an entry-scoped guard false-positives on the legal `QUAD(x) = TWICE(TWICE(x))` nesting (E-65).

4. Syntax / AST — **`openvaf/syntax/`** (**`ast.rs`**, **`generated/nodes.rs`**, **`expr_ext.rs`**, **`node_ext.rs`**, **`name.rs`**, **`validation.rs`**, **`error.rs`**)

- Typed AST nodes for every new construct above (generated from the ungrammar).
- Literal decoding (`expr_ext.rs`): based-int parsing with size/sign handling (E-46) and the mask-aware variant returning `(value, x_mask, z_mask)` for casex/casez (E-78); a compiler crash on large bare-integer-shaped literals fixed by falling back to a float constant (E-4).
- String unescaping rewritten as a single pass: the sequential `str::replace` chain corrupted overlapping escapes (`\\n`) and octal `\ddd` was missing; all consumers route through one function (E-48).
- **`Name::resolve`** ate the last character of escaped identifiers (the committed `std.va` snapshot had baked the bug in as `logi`) (E-46).

5. Elaboration — **`openvaf/hir/src/elaborate.rs`** (new file)

The compile-time flattening pass: every stage downstream of the parser is architected around exactly one flat module per artifact, so hierarchy is resolved by recursively inlining instantiated modules — alpha-renamed per instance, ports bound to the caller's nets, parameters bound to overrides — into an ordinary flat module *before* the rest of the pipeline runs. Built for module instantiation (E-5) and grown into the home of every structural feature:

- `generate for`/`genvar` unrolling (E-8), nested loops, anonymous blocks, `generate if`/`case` with elaboration- time constant folding, and the genvar-substitution fix (genvars in ordinary expressions were routed through identifier renaming, which re-escaped `1e3*(i+1)` into a broken escaped identifier; they now become literal-value holes) (E-67);
- implicit nets from undeclared instance connections, discipline derived from the connected port, prefixed per instance so no cross-instance shorts (E-41);
- hierarchical names `u1.u2.node` and `$root` via an instance-chain map and a hole scanner (E-49);
- **`defparam`** resolved through the same chain rewrite, with the LRM-mandated precedence over instance `#()` overrides (E-58);
- net concatenation in port connections expanded bit-by-bit onto vectored ports (E-59);
- bus slicing onto instance arrays and bus ports (E-5); paramset twin-module rendering (E-21); net initializers preserved through re-rendering (E-45); escaped identifiers re-rendered correctly (E-46);
- undefined instantiation targets are a hard error — they used to be silently dropped from the rendered output, turning a typo'd module name into an invisible open circuit; discipline-named misparses (`electrical out[0:2];` parses as an instantiation) and paramsets dropped over unresolvable targets get tailored messages (E-84);
- name-then-range net/port declarations (`input in[0:2]`, `electrical out[0:2]`, LRM 3.6/3.7): a textual pre-pass rewrites the `<head> <name>[range]` form to the range-then-name form (Enhancement-3), reusing all bus/port machinery; the `(`-after-range instance rule keeps instance arrays untouched (E-89). Extended to multi-name lists (`input a[0:1], b[0:3], c;`), split into one range-then-name declaration per name with per-name widths; a multi-dimensional name is left untouched (unsupported in both orders) (E-91);
- parameter-dependent declaration widths (`electrical [0:N-1] out;`, `real w[0:N-1];`): a textual pre-pass folds a declaration range whose bounds reference a parameter to a literal range, using the module's constant-integer parameter defaults (a fixpoint resolves one default through another; `from`/`exclude` constraints are skipped). Only declaration ranges (with a `:`) are folded, not bit-selects; the shared constant-integer evaluator gained an optional parameter map (empty map = the E-88 literal-only behavior). The width is fixed at the default -- a structural parameter, since OSDI has one node count per module (E-67/88 decision) (E-91). A parameter that shaped a width is then frozen to a **`localparam`** (`freeze_width_parameters`): a multi-parameter declaration is split so only the structural names freeze, and a range constraint is dropped from a frozen name. This keeps structure and behaviour consistent under a netlist override -- without it, overriding a width parameter that also bounds a runtime loop left the frozen array size behind, a silent out-of-bounds (E-92);
- the legacy **`generate <id> (start, end [, incr])`** statement (obsolete Verilog-A 1.0 analog-block loop-unroll, LRM Annex C.4) unrolled by a textual pre-pass: the index substitutes to a literal per iteration (with bit-select bracket folding, since a bus bit-select needs a literal index), constant bounds only -- a parameter bound cannot shape the unroll (E-67 scope decision) (E-88);
- **`` `__FILE__ ``**/**`` `__LINE__ ``** expanded by a textual pre-pass run before the other passes (preprocessor tokens are (kind, span) pairs into existing source and cannot carry synthesized literals): `` `__FILE__ `` becomes the root file's basename (machine-portable provenance, the E-58 rule), `` `__LINE__ `` the exact 1-based line; string/comment occurrences are skipped and replacements are inline so later line numbers stay true (E-85);
- part-select actuals sliced onto bus ports in `bind_port` (`base[msb:lsb]`, constant bounds; positional and named forms; width-1 slices onto scalar ports degrade to the bit-select) with the same

ascending bit-order convention as full-bus slicing (E-85);
- block-scoped parameter override rejected with a targeted message (a named instance override `#(.blk.p(4))` naming a hierarchical target is collected and bailed like the unknown-module errors; block-scoped parameters are local to their block and take their value from their initializer, which may reference the module's parameters) (E-87);
- hierarchical branch probes (E-86): the top module's absolute chain map rides into every inlined child so SIBLING bodies resolve `V(top.a1.b)/$root...` references; unnamed-branch forms expand to the flattened node pair; port-branch probes `I(inst.branch(<p>))` read a 0V ammeter synthesized at flattening (which also fixes child-declared `branch (<p>)` port branches, broken after inlining); hierarchy-bound monitor modules are omitted from standalone output; `ground`/net-type declarations in inlined children keep their keyword;
- **`$port_connected`** resolved at flattening time to a literal `(1)/(0)` per instance — after inlining, an open port is just a synthesized local net, so the builtin used to fail validation in exactly the unconnected case it exists to detect; top-level modules keep the native OSDI connected-flag path (E-84).

6. Definition layer — **`openvaf/hir_def/`** (+ **`openvaf/hir/`** mirrors)

`body.rs`/`body/lower.rs`/`body/pretty.rs`, `expr.rs`, `item_tree*`, `nameres*`, `builtin.rs`, `data.rs`, `types.rs`, `db.rs`, plus the new `item_tree/diagnostics.rs` and the `hir` façade (`body.rs`, `lib.rs`, `db.rs`, `diagnostics.rs`):

- statement/expression collection for every new construct: `do…while` (E-19), `repeat` and `disable` (E-9), event OR lists (E-59), concat/ replication (E-34), array aggregates and whole-array assignment (E-14), `CaseKind` + per-item don't-care masks with a stray-literal list (E-78);
- N-D array machinery: per-dimension size lists, multi-index bit-selects, per-element parameter expansion (E-15); declaration initializers on (N-D) arrays via a `Var::array_index` leaf split (E-43);
- **`localparam`** made genuinely non-overridable (it behaved exactly like `parameter`) with derived localparams still tracking inputs (E-9); paramset lowering as twin modules whose bound params are localparams with override expressions (E-21);
- name resolution: `electrical ground gnd;` ordering (E-9), nature-attribute access (`net.potential.abstol` — the third "scaffolded-but-unwired resolver boundary" found) (E-45), derived-nature resolution (E-39);
- **`builtin.rs`** (the system-function registry): the noise-table pair (E-9), the full `$random`/`$dist_*`/`$rdist_*` family (E-10), file I/O and string functions (E-11), the last fallback group (`$simprobe`, aliases, plusargs) — after which `is_unsupported()` is empty (E-12), `$simparam$str` (E-25), `$realtime` (E-59), `$table_model` (E-16);
- new diagnostics file: wrong-arity calls produced invalid HIR and crashed downstream — now a proper diagnostic (functions and parameters both) (E-43);
- multiple `analog` blocks collect into `entry_stmts` in source order — the as-if-concatenated LRM semantics, by construction (E-60);
- bus-port node ordering (`item_tree/lower.rs`): a non-ANSI header bus port (`module m(in, y); input [0:2] in;`) had its first bit merged into the header placeholder but the remaining bits appended after later ports, scrambling OSDI terminal order whenever the bus was not the last port. A pre-scan of body port widths now expands the bus into contiguous bits in header order at placeholder-creation time (E-90).

7. Types and validation — **`openvaf/hir_ty/`**

`inference.rs` (+ `inference/fmt_parser.rs`), `builtin.rs` + `builtin/generated.rs` (signature tables), `lower.rs`, `types.rs`, `validation.rs`, `validation/body.rs`, `diagnostics.rs`:

- signature tables — a recurring defect source: `ANALYSIS` made variadic (E-30); `$table_model`

given shape-synthesized varargs signatures for any dimension (a generic-varargs fallthrough *truncated* multi-arity signature lists — varargs builtins with signature lists need their own inference arm) (E-40); `TRANSITION`'s table was one argument short (3-argument calls crashed, 4-argument worked by accident) (E-47); `transition()` input typed Real, a `$dist` typo fixed (E-49); `SIMPARAM_STR` returned Real instead of String (E-25);

- format-string inference (`fmt_parser.rs`): terminates on every conversion character and reports it, so `%5d`/`%-8s`/`%08x` type their arguments correctly — previously only real conversions accepted flags/width (E-71);
- array inference: element-wise array `case`, array-literal function arguments (which previously bound nothing and silently returned 0), integer arrays typed Real, output-literal args accepted silently — all fixed (E-33); whole-array function arguments/outputs/returns (E-18, E-20, E-23); untyped function args default to Real instead of an ICE masquerading as an initializer bug (E-43);
- **`Type::base_type`** infinite loop on any array-type check (it looped on `self` instead of the cursor) — hung the compiler on simple type mismatches (E-14);
- string handling: ternary over strings (SELECT + string binary ops appended to the operator tables) (E-37);
- validation: loop bodies get their own context so the analog-operator restriction reports "loops" (LRM 4.5.1), not "conditions" (E-70); call-graph cycles among analog functions are clean errors naming the cycle instead of a recursive-inliner stack overflow (E-59); casex/casez restrictions (stray don't-care literals, `x` digits under `casez`, non-integer discriminants) (E-78); `domain discrete` with natures rejected per LRM 3.6.2.2 (E-50); parameter defaults exempt from their own ranges (the CMC "feature disabled" idiom — stock rejected diode_cmc, BSIM-CMG, PSP-HV and the HiSIM family at setup) while given values stay fully validated (E-56);
- part-selects outside port connections diagnosed (the E-78 stray-list pattern: body lowering collects them, validation reports a dedicated error naming the supported connection form) (E-85);
- contributions to port branches diagnosed (`ContributeToPortFlow`, with the declaration site labeled): `I(pb) <+ …` on a `branch (<p>) pb;` slipped through the write path unvalidated and panicked during lowering (E-84);
- contributions to an all-**`ground`** branch diagnosed (`ContributeToGround`): `V(gnd) <+ …` / `V(gnd, gnd) <+ …` reduce both endpoints to the fixed 0 reference (no unknown) and hit an `unreachable!()` in `lower_contribute_unnamed_branch` — an ICE. The write path now checks the branch's `is_gnd` nodes and reports a clean error; a real node-to-ground branch and a `V(gnd)` probe are untouched (E-97).

8. Lowering — `openvaf/hir_lower/`

`expr.rs`, `stmt.rs`, `ctx.rs`, `callbacks.rs`, `parameters.rs`, `state.rs`, `fmt.rs`, `lib.rs`:

- analog operators: `laplace_nd/np/zd/zp` via exact state-space realization (E-4) with complex pole/zero pairs per the LRM's (re, im) convention (E-31); `zi_*` via bilinear transform (E-6); `slew`/`transition` as a saturating tracking loop (E-6) with the LRM negative `max_neg_rate` convention honored (the sign defect made `slew` ignore its input and ramp away) (E-61) and a DC-singularity clamp via an `EnableIntegration` identity (E-47); `idtmod`'s modulo wrap moved off the reactive residual (it diverged) and an argument- index bug fixed (E-27); `idt` initial conditions survive into transient (E-28); the `idt` assert/reset forms with smooth reset dynamics (E-52); `limexp` kept stateless as a documented decision (E-13);
- **`$table_model`**: 1-D piecewise-linear lowered to differentiable MIR so autodiff yields the Jacobian for free (E-16); N-D multilinear as recursive 1-D (E-17, E-40); natural cubic splines with the precomputed moment matrix (E-22);
- events: `cross`/`above`/`timer` with persistent previous-value slots (E-8); OR lists folded with a select-based bool-or (a raw `ior` ICEs const-eval) (E-59); final-step gating and phase-list AND-ing (E-53);
- variable persistence: genuine per-instance `hidden_state` storage behind a two-pass MIR build

(E-7), typed slots so integer state stopped crashing instruction selection (E-32);

- callbacks (`callbacks.rs`, `fmt.rs`): the `$display` family with the full `[flags][width][.precision]` surface and `%h/%b/%r` translation (E-71); file I/O and string formatting through a generalized `PrintDst` sink (E-11); the deterministic seed-and-salt RNG lowering (E-10); `$simparam$str` (E-25); simulation-control return flags (E-55); `$discontinuity` (E-24);
- operator fixes: unary ~ emitted arithmetic negate; the constant folder sign-extended >> where the runtime was unsigned — const and runtime disagreed (E-37);
- whole-array coercion (`expr.rs`, `stmt.rs`): inference records a coercion of a whole array as a cast on the array *expression*, but `lower_array_elems_impl` decomposes the array and lowers each element itself, so `lower_expr`'s `needs_cast()` never saw it — the cast was silently dead, and every new array-consuming context re-inherited the same crash (an integer element reaching a float MIR op, which const-eval has no case for: *"invalid operation fdiv Int(1) Float(..)"*). Patched per-call-site four times — the `case` discriminant (E-33), `laplace_*/zi_*` integer-literal then integer array-variable coefficients, and an integer `case` item against a real discriminant — before being fixed at the chokepoint, which now honours the recorded cast for every consumer (E-214). The explicit `coerce_real` flag remains for consumers whose element type is fixed by the language rather than by an inferred cast (a coefficient vector is real per LRM 9.19, and inference records no cast for it);
- masked casex/casez comparisons (two `iands` ahead of the existing integer equality) (E-78); array function-argument writeback (E-20) and array returns via a function-named var-array (E-23);
- named port branches (`branch (<p>) pb;` — LRM 3.7.2): a flow probe of a PortFlow-kind branch routes through the same `CurrentKind::Port` param as a direct `I(<p>)`, so E-29's defining equation covers both spellings; probing one used to hit `unreachable!()` (E-84);
- literal **`if`** conditions lower only the taken branch — a dead analog operator (`transition()` under `if ((0))`, the exact shape the `$port_connected` rewrite produces) survived const-folding as a detached-but-interned op whose state setup read optimized-away values and aborted codegen (E-84);
- **`case`** fall-through block left unsealed: `max`/`min`/`abs` lower through `make_cond` to real control flow, so one in a `case` default arm moved the builder onto its own merge block — the seal at the end of the case landed there and the case's own fall-through block was never sealed ("block N is not sealed"). It is now sealed where it is created; the branch just emitted is its only predecessor (E-291).

9. MIR core and optimization — **`openvaf/mir*`**

`mir/` (`instructions.rs` + `instructions/generated.rs`, `dfg.rs` with `dfg/phis.rs` and `dfg/uses.rs`, `dominators.rs`, `flowgraph.rs` with `flowgraph/transversal.rs`, `layout.rs`, `cursor.rs`, `lib.rs`, `builder/generated.rs`), `mir_opt/` (`simplify_cfg.rs`, `simplify.rs`, `const_eval.rs`, `global_value_numbering.rs`), `mir_autodiff/` (`builder.rs`, `lib.rs`, `live_derivatives.rs`), `mir_llvm/` (`builder.rs`, `intrinsics.rs`), `mir_build/`, `mir_interpret/`:

- three pre-existing general CFG bugs, all found via event-counter verification: a dangling-reference bug in unreachable-block removal, a multi-exit post-dominance bug in the dominator-tree builder, and a block-merge bug that could corrupt which block the DAE builder treated as the exit (E-8);
- the post-dominator sink-root pathology that let dead-code elimination delete op-dependent `$fatal` calls (side-effecting callbacks under op-dependent control now stay in eval; control dependence computed by arm-reachability) (E-55);
- `split_tainted.rs` tolerates layout-detached branch instructions (a branch whose condition const-folded has no CFG effect to taint; it used to unwrap and panic) (E-84);
- const-eval agreement with runtime semantics for shifts and the bitwise-not fix (E-37);
- autodiff: noise operators process before any `ddt` (a shared ddt evaluated between two noise operators silently dropped the second source), and the reactive coupling of a `ddt`-shaped noise wave is registered so small-signal pruning keeps it (E-54);
- **`llvm.fabs.f64`** registered in the LLVM intrinsics table — it never had been, and the signed noise-

power convention needed it (E-42);
- new opcodes/instructions supporting the operator work (barrier/ integration-identity forms; MIR deliberately has no `fabs`, so lowering composes it) (E-47, E-52);
- **`const_eval.rs`** — folding must match codegen or decline: `eval_binary` evaluated `5/0`, `5%0` and `i32::MIN/-1` *inside the compiler*, so a literal zero divisor killed it outright while a runtime one had always been accepted. It now returns `Option`, declines the undefined cases and out-of-range shifts, and folds +/−/∗ with wrapping arithmetic to match the emitted code (E-286);
- **`simplify_cfg.rs`** — a const-folded branch did not flag its change, so the sweep that collects orphaned blocks never re-ran and a phi kept an edge naming a value reachable only through the deleted edge — a broken-SSA function the release build carried forward, the MIR verifier being a `debug_assert!` (E-287);
- **`mir_llvm/intrinsics.rs`** — two declarations disagreed with the calls emitted: `hypot` was declared with one parameter and called with two (E-288), and the overloaded `llvm.ctlz` was registered without its `.i32` type suffix — it backs `$clog2` (E-289). Both produced modules the LLVM verifier rejects, and both were invisible in release for the same reason.
- branch-to-jump rewrites must retire the condition's use: a `Branch` carries one value operand, a `Jump` none, so overwriting the instruction in place leaves the condition's use record naming an operand that no longer exists. `simplify_bb`'s empty-exit-block rewrite and `dead_code_aggressive`'s dead-block rewrite both did; the two in `const_fold_terminator` already zapped/detached first (E-294).

10. DAE construction — `openvaf/sim_back/`

`dae.rs` + `dae/builder.rs`, `topology*` (incl. `lineralize.rs`, `small_signal_network.rs`), `noise.rs`, `context.rs`, `lib.rs`:

- implicit equations for indirect branch assignment (one new unknown
  - residual per statement) (E-2) and the absdelay synthetic two-node form (E-1) — with equation indices kept stable by mapping dead/collapsed unknowns to zero-contribution placeholders instead of renumbering;
- port-flow probes **`I(<port>)`**: a TODO stub returning 0 became a synthesized DAE unknown mirroring the node's KCL residual (E-29);
- probe-only branches: probing a never-contributed branch read 0 and an open circuit — a 0 V-source pass materializes them, enabling ideal ammeters and flow-only signal-flow disciplines (E-36);
- noise factors (`noise.rs`): equation-path noise was silently lost (never attached to the DAE); factors generalize to `re + jω·im` with no extra matrix unknowns (E-54); same-named source grouping metadata (E-42);
- `idt` reset-mode charge dynamics and conditional step bounding (E-52).
- voltage-source branches feeding internal nodes were open circuits at DC: the small-signal (noise/`ac_stim`) pruner keyed node registration on the branch's LIVE voltage unknown, so a pure `V(a, f) <+ expr` (V(a,f) never read) registered nothing and the node's conduction silently moved to the AC-only residual; any voltage-capable branch now disqualifies its nodes (E-86);
- probed **`V(x,y) <+ 0`** branches were node-collapsed away, making `I(branch)` read zero; hint pairs whose branch current is a DAE unknown are suppressed (the unprobed collapse idiom is untouched) and `NodeCollapse::hint` tolerates suppressed pairs; pinned by the permanent `vsrc_internal_node` snapshot test. Behavior change: a collapsible branch whose current the model references (MVSG_CMC's access resistances carry noise on `flow(d,drc)`) stays a real 0V source — electrically identical, two extra matrix rows (E-86);

- **`lineralize.rs`** — one analog operator nested directly inside another (`ddt(ddt(x))`): an operator materialized as an implicit equation deletes its own instruction, but a later linear contribution holds its dimension values *outside* the data-flow graph where `replace_uses` cannot reach them — and with direct nesting that stored dimension IS the deleted result. Pending entries are now retargeted onto the implicit unknown the inner operator became, which is also the correct second-derivative formulation (E-293);
- **`small_signal_network.rs`** — pruning is best-effort: the linearity classifier and the dimension replay can disagree, and the pass then indexed a `val_map` key that was never inserted ("no entry found for key"). It now gives up on that value — resolving its placeholder so no invalid value survives — instead of crashing (E-292).

11. OSDI code generation — **`openvaf/osdi/`**

`lib.rs`, `metadata.rs` + `metadata/osdi_0_4.rs`, `inst_data.rs`, `eval.rs`, `load.rs`, `setup.rs`, `compilation_unit.rs`, `ndatable.rs`, `stdlib.c`, `build.rs`, reference headers:

- descriptor emission for every ABI evolution (see the ngspice report's ABI table): the absdelay/last_crossing info arrays (E-1, E-6), `OsdiNode.nodeset` (E-45), the `ac_stim` source array + `load_ac_stim` (E-51), stride-2 signed noise pairs in `load_noise` (E-54) — the version tuple lives in `osdi/src/lib.rs`;
- **`metadata.rs`**: the `PARA_FLAG_FIXED` parameter-descriptor flag (a free bit of the `flags` field, additive — no layout/ABI change) set on every `localparam`, so the simulator can warn when a netlist tries to set a non-settable parameter (a `localparam`, or a structural width parameter frozen by Enhancement-92) (E-93);
- **`eval.rs`**: final-step flag gating (E-53); simulation-control return flags (E-55); the `ac_stim` large-signal value (was an `unreachable!()` panic) (E-26);
- **`inst_data.rs`**: absdelay slot layout (E-1); hidden-state slots typed by the variable (`ty_f64` was hardcoded — integer state fed f64 loads into integer MIR ops and aborted instruction selection) (E-32);
- **`compilation_unit.rs`** (print codegen): the `%b` segfault — the binary-formatted string was remembered for `free()` but never pushed to the `snprintf` argument list, so the matching `%s` consumed a garbage pointer (E-71); the print-callback machinery with its shadowed-`fun` and volatile-table IPO traps (E-11);
- **`stdlib.c`**: the file-descriptor table behind `$fopen`/`$fdisplay`/... (E-11); the splitmix64 `osdi_rng_*` runtime (E-10); a `simparam_str` loop/return bug (E-25);
- **`ndatable.rs`**: `ddt`/`idt` natures resolved through `resolve_nature_index` instead of panicking on derived natures (E-39);
- **`setup.rs`**: range validation with default-exemption (E-56); a 64-line abandoned unsafe experiment (`VoidAbortCallback`) deleted in the warning cleanup (E-66);
- **`build.rs`**: the copied-target stdlib compilation trap fixed (E-10);
- **`inst_data.rs`**: **`$temperature`** read as an operator ARGUMENT used the wrong struct-GEP type — the field type (`double`) instead of the instance-data struct — so LLVM computed the offset as a flat `5*sizeof(double)` rather than `offsetof(instance, temperature)`. The shipped compiler died with SIGSEGV on `ac_stim("ac", $temperature, 0)`; the same wrong type was on the operating-point-variable read path (`nth_opvar_ptr`) (E-290).

12. Driver, libraries, and infrastructure

- `openvaf-driver/src/main.rs`: hard errors now exit non-zero — the error arm printed the failure but fell through to a success exit, so every elaboration failure looked like a successful compile to shell scripts (a quirk first noticed during E-58's work) (E-84);
- `openvaf-driver/src/crash_report.rs`, `linker/src/lib.rs`, `lib/bforest/` (`map.rs`, `pool.rs`, `set.rs`), `lib/typed_indexmap/` (`set.rs`): the zero-warning cleanup (44 → 0 across

13 crates: lifetime annotations, dead code, nightly cfgs, an unused LLVM-prefix probe) (E-66);
- `basedb/` (`ast_id_map.rs`, `lints.rs`, `lib.rs`, `diagnostics/preprocessor_error.rs`, `diagnostics/syntax_error.rs`): AST-id plumbing for elaboration-created items (the `AstId::from_erased` placeholder used by array returns; E-23), preprocessor/syntax diagnostic rendering for the new errors;
- `.llvm-version` pins the LLVM 18 toolchain expectation; the `hir` and `preprocessor` `Cargo.toml`s carry the dependency additions their new code needed.

13. The test suite — **`openvaf/openvaf/tests/`**, **`lib/mini_harness/`**, **`test_data/`**, **`integration_tests/`**

The fork's own integration suite over real compact models had never run in this project: its OSDI loader was frozen at ABI 0.4, its optional VACASK legs panicked on a never-initialized submodule, and the tests hide behind `RUN_DEV_TESTS=1`. Fixed test-side only (E-68):

- `tests/load/osdi_0_4.rs` + `load/mod.rs`: loader structs synced to ABI 0.7; `mock_sim/mod.rs`: stride-2 noise convention; `mini_harness`: missing directories skip with a note instead of panicking;
- VACASK removed entirely (`.gitmodules`, the submodule, 34 harness legs and their stale snapshots): AGPL-3.0 cannot be vendored here;
- `test_data/` (95 files as a class): new fixtures for new features and snapshots regenerated whenever descriptor contents legitimately changed — every regeneration reviewed (the diffs are the project's own features: `flow(<port>)` unknowns, probe-only branches, implicit-equation nodes); `integration_tests/*/*.osdi` regenerated at each ABI bump.

- Two correctness blind spots closed with mutation-tested guards (E-295, verification-only): the full 4x4 multi-terminal conductance AND capacitance matrices (the autodiff suite was 2-terminal plus one off-diagonal, so the KCL-derived source row and the zero body row were untested), and per-instance parameter-slot readback across 13 interleaved model/instance parameters (the guard for the E-290 offset class) (E-295).

14. Per-enhancement index (compiler-touching enhancements)

E-1 *sim_back, osdi*

`absdelay()` synthetic-node DAE + descriptor arrays

E-2 *parser→hir_lower, sim_back*

indirect branch assignment (implicit equations)

E-3 *parser, hir_def*

vectored (bus) nets with bit-selects

E-4 *hir_lower, syntax, parser*

`laplace_*` state-space + array-literal parsing

E-5 *elaborate.rs (new), parser*

module instantiation via compile-time flattening

E-6 *preprocessor, parser, hir_lower, osdi*

directives, shifts, slew/transition, zi_*, last_crossing

E-7 *hir_lower (state), osdi*

`@(initial_step)` gating + variable persistence

E-8 *parser, elaborate, hir_lower, mir/mir_opt*

generate for + events; three general CFG bug fixes

E-9 *hir_def, hir_lower, sim_back*

noise tables; localparam/ground/strings; repeat/disable

E-10 *hir_lower, osdi stdlib*

`\$random`/`\$dist_*` deterministic RNG

E-11 *hir_lower, osdi*

file I/O + string functions (PrintDst)

E-12 *hir_def, hir_lower*

last unsupported builtins as LRM fallbacks

E-13 *hir_lower*

`limexp` kept stateless (documented decision)

E-14 *hir_def, hir_ty, hir_lower*

array literals/params/dynamic indexing; base_type hang

E-15 *hir_def, hir_ty, hir_lower*

N-D arrays

E-16/17/22/40 *hir_ty, hir_lower*

`\$table_model`: 1-D, N-D multilinear, cubic spline, varargs sigs

E-18/20/23/33 *hir_ty, hir_lower, basedb*

arrays in analog functions: in/out/return/literals + array case

E-19 *tokens→hir_lower*

`do … while`

E-21/44 *parser, elaborate, hir_def, sim_back*

paramset + hidden system parameters

E-24/55 *hir_lower, mir, osdi*

`\$discontinuity`; `\$finish`/`\$stop`/`\$fatal` honored

E-25 *hir_ty, osdi stdlib*

`\$simparam\$str`

E-26/51 *osdi, sim_back*

`ac_stim`: crash fix, then full AC-RHS injection

E-27/28/52 *hir_lower, sim_back*

the `idt` family: modulo, IC, assert/reset

E-29/36 *sim_back*

port-flow probes; probe-only branches

E-30 *hir_ty, hir_lower*

variadic `analysis()`

E-31 *hir_lower*

complex poles/zeros in root forms

E-32 *osdi inst_data*

integer persistent state (crash fix)

E-34 *tokens→hir_lower*

`{…}` concat + `{n{…}}` replication

E-35 *lexer*

EOF comment hang

E-37/38 *hir_lower, mir_opt, parser*

operator + precedence audits (~, >> fold, % level)

E-39 *parser, hir_ty, osdi*

derived natures unwired at the node-kind boundary

E-41/49/58/59 *elaborate*

implicit nets; hierarchical names; defparam; port concat + OR lists

E-42/54 *sim_back noise, mir_autodiff, mir_llvm, osdi*

correlated + node-free complex noise factors

E-43 *hir_def (+diagnostics), hir_ty*

initializers completed; arity diagnostics

E-45 *osdi, hir_def, elaborate*

nodesets + nature-attribute access (ABI 0.5)

E-46/48 *lexer, syntax*

escaped ids, based literals, single-pass unescaper

E-47 *preprocessor, hir_ty, hir_lower*

`` `default_transition `` + transition fixes

E-50 *hir_ty validation*

domain-binding validation

E-53 *hir_lower, osdi eval*

`@(final_step)` + phase lists

E-56 *hir_ty, osdi setup*

parameter defaults exempt from ranges

E-59 *tokens→hir_lower, hir_ty*

`\$realtime`; OR lists; recursion diagnostics

E-61 *hir_lower*

`slew` sign fix (operator-args audit)

E-65 *preprocessor*

macro-recursion guard

E-66 *13 crates*

zero-warning cleanup (44 → 0)

E-67 *tokens, parser, syntax, elaborate*

generate audit: genvar fix, nesting, if/case

E-68 *tests, mini_harness*

integration suite enabled; VACASK removed

E-70 *hir_ty validation*

loop-context diagnostics

E-71 *hir_ty fmt, hir_lower fmt, osdi*

display-format surface + `%b` segfault

E-78 *lexer→hir_lower, hir_ty*

`casex`/`casez` don't-care masks

E-84 *parser, elaborate, hir_ty, hir_lower, mir_opt, driver*

LRM example sweep: 6 defect fixes (port-branch panic, parser robustness, silent undefined modules, `\$port_connected` on open ports, dead-op codegen, exit codes)

E-85 *parser, elaborate, hir_def, hir_ty*

`` `__FILE__ ``/ `` `__LINE__ `` + connection part-selects (the last two sweep findings)

E-86 *parser, elaborate, sim_back*

hierarchical branch probes + 2 DAE fixes (V-source-to-internal open circuit, collapse-of-probed-branch)

E-87 *parser, elaborate*

block-scoped parameters (validated) + clean diagnostic for the illegal `#(.blk.p())` override

E-88 *elaborate*

legacy `generate <id> (start,end)` analog-block loop-unroll (textual pre-pass)

E-89 *elaborate*

name-then-range net/port decls (textual normalize) + Annex E SPICE-primitives example library

E-90 *hir_def (item_tree)*

multi-bit input bus port bit reads: pre-scan body widths so a non-last bus port's bits stay contiguous in header/terminal order

E-91 *elaborate*

multi-name name-then-range declarations + parameter-dependent declaration widths (folded from the parameter default)

E-92 *elaborate*

freeze structural (width) parameters to `localparam` (split multi-parameter decls) so a netlist override cannot desync the frozen width from behaviour

E-93 *osdi (metadata)*

flag `localparam` OSDI parameters non-settable (`PARA_FLAG_FIXED`, additive) so ngspice can warn on a netlist override (ngspice side too)

E-96 *parser (module grammar)*

parse a module-level `generate for/if/case` without the optional `generate`/`endgenerate` keywords

E-97 *hir_ty (validation)*

clean diagnostic (was an ICE) for a contribution to an all-`ground` branch (`V(gnd) <+ …`)

E-101 *hir_ty (builtin), mir_opt / mir_interpret / mir_llvm*

`\$clog2`: accept one argument (`INT_MATH_1`, was a 2-arg signature) and compute `ceil(log2 n)` = `bit_width(n-1)` in all three backends (was `floor(log2 n)+1`, wrong on exact powers of two)

E-102 *parser (items), syntax (ungrammar + ast), hir_def (item_tree)*

name-then-range array-valued parameters (`parameter real c[0:2]`); `parameter()` accepts post-name `[range]`, `lower_param` resolves dims per name

E-103 *mir_llvm (intrinsics)*

register the `llvm.ceil.f64` intrinsic (was missing; `ceil()` of a non-constant argument crashed codegen -- `floor` was registered)

E-104 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr)*

add `\$rtoi` (real->int, truncate toward zero via `ficast((x<0)?ceil:floor)`) and `\$itor` (int->real) conversion builtins

E-105 *hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)*

`\$sscanf`/`\$fscanf` honour the format conversion base -- parse the format string, dispatch integer fields to base-specific scanners (`%h`/`%x` hex, `%o` octal, `%b` binary)

E-106 *hir_ty (types, inference), hir (signatures), hir_lower (expr, callbacks), osdi (compilation_unit, stdlib.c)*

string relational comparison (</<=/>/>=) via a lexicographic `osdi_strcmp` callback (`RELATIONAL_COMPARISON` adds the STR signature; `a<op>b` lowers to `strcmp(a,b)<op>0`)

E-107 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)*

add `\$fgetc(fd)` single-character read as a `FileOp::Getc` -> `osdi_fgetc` (completes the file I/O family)

E-108 *syntax (name), hir_def (builtin), hir_ty (builtin), hir_lower (expr, callbacks), osdi (stdlib.c)*

add `\$ungetc(c, fd)` one-character pushback as a 2-arg `FileOp::Ungetc` -> `osdi_ungetc` (companion to `\$fgetc`)

E-109 *hir_lower (callbacks), osdi (load)*

correct `noise_table` (piecewise-linear in `f`) and `noise_table_log` (log-log, `P=10^lerp(log10 p, log10 f)`) interpolation to LRM 4.6.4.3/4.6.4.4 -- both were nonconformant (lin-log, and a mis-keyed log-freq input)

E-147 *hir_ty (validation/body)*

fix exponential-time compilation of nested **`?:`**. Found by a robustness campaign (117 production CMC models + ~50 adversarial inputs + 4000 mutation-fuzz iterations). The body validator's `validate_expr` ends by recursing into children via `walk_child_exprs`; arms that validate their own operands `return` first (`Call`, `Path`), but the `Select` (ternary) arm did NOT -- it validated `cond/then_val/else_val` via `validate_condition` and then fell through, validating `then_val/else_val` a SECOND time, so a chain of N nested `?:` was validated 2^N times. Depth ~30 (reachable via macros) hung the compiler; `sample`-profiling confirmed an unbounded `validate_expr↔validate_condition` recursion with doubling call count. Fix: add `return;` to the `Select` arm (the operands are already fully validated). O(2^N)→O(N): depth 40 hang→0.09 s, depth 2000→1.6 s. Behaviour-preserving: all 117 models give the IDENTICAL verdict before/after (0 flips, head-to-head), BSIM4 (12.6k lines) still ~2.3 s, and hir_ty/hir/hir_lower/sim_back tests pass with no snapshot changes. Campaign also confirmed 0 panics/segfaults across 4000 fuzz iterations; remaining lower-severity findings (parser stack overflow on ~4k-32k-deep nesting, `include` self-recursion, huge array dimension) are documented follow-ups. Verified 7/7 (nested_cond_examples: depth 20/40/80/160 all <0.2 s, linear growth, correct piecewise value in ngspice)

E-148 *parser (parser, grammar/expressions), preprocessor (diagnostics, processor), basedb (diagnostics/preprocessor_error), hir_def (item_tree, item_tree/diagnostics, item_tree/lower), hir (elaborate)*

compiler hardening: closes the three lower-severity robustness findings from E-147 (pathological input → crash/hang) by turning each into a clean bounded diagnostic. (1) Parser expression-depth limit: a shared `Parser::expr_depth` counter, incremented on each `atom_expr` (recursion) and per operator in the `expr_bp` loop (operator-chain / tree depth), bounds total expression-tree depth at 1000 and recovers to the next statement boundary when exceeded — so `----…x` / `x+1+1+…` / `((…))` / `sin(sin(…))` / `1?…:0` no longer overflow the recursive-descent parser OR a later recursive tree traversal (`expr_bp` split into a save/restore wrapper + `expr_bp_inner`; `atom_expr` into an inc/check/dec wrapper + `atom_expr_inner`; reuses the existing `err_recover/unexpected_tokens_msg` recovery). (2) **`` `include ``** recursion cap: an `include_depth` counter on the preprocessor's `Processor` caps nesting at 64, emitting a new `PreprocessorDiagnostic::IncludeRecursionLimit` (rendered in basedb) instead of overflowing the stack on a self-including file — mirrors the E-65 macro-recursion guard. (3) Array element cap: a shared `array_elem_count` helper (product of `|msb−lsb|+1`, capped at 2^20 ≈ 1.05M) + a new `ItemTreeDiagnostic::ArrayTooLarge`, applied at EVERY expansion site — variable arrays, parameter arrays, net/port buses, array function returns (all in `hir_def` item-tree `lower.rs`), and instance arrays in BOTH the item-tree lowering and the `hir` elaboration pass (which re-expands `dev s[0:N]()` into rendered text) — so `real x[0:100000000]` (and param/bus/instance equivalents) degrade to a single scalar with an error instead of exhausting memory. Behaviour-preserving: all 117 production CMC models give the IDENTICAL verdict before/after (0 flips, head-to-head), BSIM4 still ~2.3s; parser/preprocessor/hir_def/hir_ty/sim_back unit tests pass with no snapshot changes; 0 build warnings. Verified 17/17 (robustness_examples: 5 deep-expression shapes + self-include + 4 huge-array kinds all error in <1s no crash/hang with expected diagnostics; valid nested-ternary-30 / parens-100 / 100-term-sum / small arrays still compile). Fully resolves the E-147 campaign's remaining findings

E-185 *mir_autodiff (builder.rs)*

autodiff audit: hypot & atan2 derivative fixes. A deep audit compared the AC small-signal conductance dI/dV (built by the mir_autodiff automatic differentiation that feeds AC/convergence/noise/

pz) of a battery of nonlinear laws I=f(V) against the ANALYTIC f'(V), and caught two builtins right in VALUE (DC correct) but wrong in DERIVATIVE -- the accidental-correctness pattern, first on the compiler side. (1) hypot(x,y): the autodiff rule computed (x'+y')/(2*hypot) -- the sqrt(x) pattern x'/(2*sqrt) misapplied to a 2-arg function -- instead of the correct (x*x'+y*y')/hypot; 28% off at V=0.7, y=0.5, only accidentally correct at x=0.5. Fixed by splitting hypot from sqrt in inst_cache (cache holds hypot(x,y), not 2*hypot) and a correct chain rule with the usual zero/one folding. (2) atan2(x,y): two bugs in the cached factors feeding the shared Pow|Atan2 chain rule (x'*c0 + y'*c1)*c2 -- the common factor c2 was (x^2+y^2) where the rule MULTIPLIES, so it needed the reciprocal 1/ (x^2+y^2); and c1 (the y' factor) was +x where d/du atan2 = (x'*y - y'*x)/(x^2+y^2) SUBTRACTS, so it needed -x. Wrong magnitude AND sign. Fixed: c2 = 1/(x^2+y^2), c1 = -x. The fix is at the autodiff-rule level so it applies to resistive currents, reactive charges (verified via AC susceptance of ddt(hypot)), higher-order derivatives, and ddx. verify_vafautodiff: 9 checks (both arg orders, sqrt-form equivalence, the V=0.5 accidental-correctness point, atan2 sign + atan2(V,V)->0 cancellation, reactive susceptance) + a 15-builtin regression battery confirming the rest were already correct. Regression 150/150.

E-187 *mir_opt (simplify.rs)*

math-identity simplifier: invalid inverse-function cancellations. The algebraic simplifier (simplify_unary_op) rewrites f(g(x)) -> x for function-inverse pairs. Six of these were applied unconditionally even though the cancellation is only valid when f is a true left inverse of g over ALL of g's range -- so they returned the raw inner x, WRONG for ordinary finite inputs, corrupting the DC value itself (a more severe class than the derivative-only autodiff bugs). (1) asin(sin(x)) != x for |x|>pi/2 (asin(sin 3)=pi-3=0.1416). (2) acos(cos(x)) != x outside [0,pi] (acos(cos 4)=2pi-4). (3) atan(tan(x)) != x for |x|>pi/2 -- and this is a legitimate angle-WRAP idiom the optimizer silently defeated (atan(tan 2)=2-pi). (4) acosh(cosh(x)) != x for x<0 (cosh even -> |x|). (5,6) sqrt(x*x) *and sqrt(x**2) are |x|, not x (sqrt((-3)^2)=3, returned -3). The const-fold path (eval_ unary) evaluates each op numerically and was always correct, so constant spot-checks passed -- the bug lived only in the symbolic cancellation on runtime values. Fix: Asin/Acos/Atan/Acosh decline to simplify (return None); the Sqrt arm returns None (MIR has no fabs, so the sqrt stays and computes |x|). Kept the cancellations valid over the whole real line (tan(atan), ln(exp), asinh(sinh), atanh(tanh), sinh(asinh), cosh(acosh), log10(pow(10,.))). Rewrites unsound only for Inf/NaN (x-x->0,* x*0->0, sin(asin(x)), exp(ln(x))) left as standard finite-math. Removed the now-unused F_TWO import. mir/mir_opt/mir_autodiff unit tests unchanged (13/7/19). New example mathident_ examples/verify_mathident.py: 12 DC-value checks. Regression 151/151.

E-186 *mir_autodiff (builder.rs + lib.rs)*

autodiff audit: real-modulo (%) derivative fix. The same AC-conductance referee (E-185) caught a third builtin right in VALUE, wrong in DERIVATIVE. Verilog-A real modulo lowers to the Frem opcode; x % c = x - floor(x/c)*c *is a slope-1 sawtooth in x, so d/dx(x % c) = 1 away from the wrap points -- yet Frem was grouped with the genuinely-constant opcodes (floor, ceil, $clog2, integer/bitwise ops, comparisons) and forced to derivative 0 in TWO places: (1) the live-derivative gate zero_derivative( ) in lib.rs, which decides which SSA values even depend on the differentiation variable -- with Frem listed, a modulo result was declared independent of the unknown so its derivative was never even requested; and (2) the inst_derivative chain rule in builder.rs (the => return no-edge arm). Because the gate came first the AC/Jacobian contribution of any modulo term was identically zero: a .dc sweep of I=1e-3*(V % 1.0) had slope exactly 1e-3 while the AC conductance read 0. Fixed by removing Frem from both zero-derivative groups and giving it the correct rule d/du(x % c) = x' - floor(x/c)*c' -- folding to just the dividend derivative x' for the common constant divisor (c'=0), and emitting the -floor(x/c)*c' *term only when the divisor itself carries a derivative. floor/ceil/$clog2/integer ops correctly stay in the zero group. Applies everywhere the derivative is taken -- resistive I, reactive Q, ddx, higher orders. verify_vafautodiff extended to 14 checks (adds 5 modulo cases: two constant divisors, prefactor scaling, chain-rule d/dV (V%1)^2 = 2*(V%1), and floor/ceil staying 0), a separate 3-terminal probe confirming the variable-divisor branch, cross-checked via the ddx value path;

the 19 mir_autodiff unit tests are unchanged and pass. Regression 150/150.

E-213 *syntax (lib.rs, parsing/tree_builder.rs), lexer (lib.rs), parser (grammar/paths.rs), preprocessor (parser.rs), examples/vafcrash_examples/**

crash hardening: four compiler panics. Fuzzing openvaf-r with malformed Verilog-A found four distinct panics: instead of printing a diagnostic the compiler aborted with "OpenVAF encountered a problem and has crashed!" (exit 101) and directed the user to file a bug report -- for what is often just a typo. All are reachable from ordinary source mistakes; the headline case is a module merely missing its `endmodule`, one of the most common Verilog-A editing errors. BUG 1 -- EOF SPAN (tree_builder.rs): a parse error at end of file built its span as TextRange::at(self.text_ pos,); text_pos is already the end of the source, so adding the previous token's length produced a span running PAST the end of the file -- 6..12 for the 6-byte input "module". Mapping it back to its file then failed assert!(range.end() <= self.range.end()) in FileSpan::with_subrange (preprocessor/ sourcemap.rs), crashing the compiler WHILE TRYING TO PRINT THE ERROR ("subrange 6..12 -> 6..12 must fit into the total range 0..6"). FIX: an empty range at EOF -- exactly what the adjacent expected_at field already does. Also find_ctx_range (syntax/lib.rs) maps a position through half-open [start,end) ranges that never cover the EOF position itself (pos == last_range.end() compares Less against every range); it .expect()ed and panicked, now clamps. Triggers: missing endmodule; bare `module`; module header then EOF; unclosed `analog begin`; unterminated string. BUG 2 -- REAL LITERAL (lexer/lib.rs): e/E was consumed as an exponent marker without checking an exponent follows (eat_float_exponent()'s return, which reports whether a digit was seen, was discarded), so `1e` became a Float token whose text does not parse as f64 and panicked in ast::StdRealNumber::value()'s src.parse().unwrap(). FIX: an `e` only joins the number when a digit (optionally signed) follows; a bare 1e lexes as 1 + identifier e and surfaces as an ordinary parse error -- the approach based_literal_body already takes for a malformed 8'squark. Valid exponents (1.5e3, 2e-3, 1E6) untouched. Triggers: 1e, 1e+, 99e, 1.5e, parameter real p=2e. BUG 3 -- PREPROCESSOR EOF (preprocessor/parser.rs): previous_range() (self.full_tokens[pos], reached while building the "expected an identifier" diagnostic) and followed_by_bracket_without_space( ) (self.relevant_tokens[self.pos + 1u32]; "index out of bounds: the len is 2 but the index is 2") indexed the token list directly, out of bounds one past the last token -- a bare `"define" ending the file. FIX: both use .get() with a sensible fallback, mirroring the .get( ).map_or() idiom current_range() already used. BUG 4 -- PATH() ASSERT (parser/ grammar/paths.rs): path() opened with assert!(p.at_ts(PATH_SEGMENT_TS)), a precondition several callers do not check and plausible input violates: aliasparam x = 5;(a literal where a parameter name belongs),aliasparam x = ;, I(<1>)/I(<>)(port_flow),discipline d 1 = 2;. FIX: drop the assert -- the next line, p.expect_ts(PATH_SEGMENT_TS), already emits "expected identifier" and returns false, so the error is reported and an empty path node completed (nature's parser guarded its own call site this way in E-39; this makes the helper safe for every caller). This is the openvaf-r counterpart to E-212 (the same campaign against ngspice) and a direct continuation of E-148: E-148 hardened against pathological DEPTH (inputs too big), E-213 covers inputs that stop too early or are malformed. No change to any accepted program -- every fix is on a path that previously aborted the compiler; generated OSDI for every existing model is identical. Verify (examples/vafcrash, 25 checks): every repro yields a clean error instead of a crash; valid code unchanged (resistor; real exponents; aliasparam bound to a real parameter; define with args). NOTE: openvaf-r installs a CUSTOM panic hook printing its own message and exiting 101 rather than dying on a signal or printing the usual Rust "panicked at" -- a crash check looking only for those two (as E-148's suite does) scores these panics as ordinary errors, so the new suite treats exit 101 and the hook message as a crash. Toolchain tests unchanged (lexer 8, preprocessor 6, basedb 9, sim_back 26, hir 15, hir_lower 4). 25 checks. Regression 173/173 (new example folder).

E-214 *hir_lower (expr.rs, stmt.rs), examples/arraycast_examples/**

whole-array type coercion: a recurring crash class, fixed at its root. An integer Value reaching a float MIR op (feq/fmul/fsub/fdiv) panics mir_opt::const_eval::eval_binary, which has no (Int, Float) case -- "invalid operation fdiv Int(1) Float(..)", exit 101, "please open an issue". (Unfolded, the same defect instead reaches LLVM as `i32 = fadd .., ConstantFP:f64` and aborts with "LLVM ERROR: Cannot select" -- one bug, two faces.) FOUR INSTANCES, each patched at its own call site as found: the `case` discriminant over an integer array (E-33, element type hardcoded real); laplace_/zi_ integer-LITERAL coefficients (b77266ec); laplace_/zi_ integer ARRAY-VARIABLE coefficients (c55812d6 -- the previous fix had reasoned "a whole-array variable reference is already real by its declaration", which is false for an `integer` array); and an integer `case` ITEM against a REAL array discriminant (8d0ab057 -- the opcode is chosen from the discriminant, Feq, but the item's elements lower to i32). ROOT CAUSE: inference DOES record the coercion (expect( ) -> casts.insert(item, Array{Real})), but it lands on the array EXPRESSION, and lower_array_ elems_impl decomposes the array and lowers each element itself -- so lower_expr's needs_cast( ) never saw it and the cast was silently DEAD. Every new array-consuming context therefore re-inherited the trap. FIX: lower_array_elems_impl -- the one place every whole-array consumer passes through -- now honours a recorded cast to a real target (coerce_real |= matches!(needs_ cast(expr), Some((_, dst)) if *dst.base_type() == Type::Real)), making inference's intent effective for all consumers instead of requiring each call site to ask. Proven in isolation: with the case-site flag disabled the structural guard alone fixes every case repro. lower_case additionally keeps its own discr_op == Opcode::Feq coercion -- choosing the opcode from the discriminant makes matching operand types that function's invariant, independent of inference's bookkeeping. NO MISCOMPILE: the integer spelling of a filter is bit-identical to the '{1.0} spelling (worst AC diff exactly 0 dB over 13 points) and matches the analytic response to 3e-8 dB; an integer `case` item still selects its arm. MUTATION-TESTED: reverting the fixes makes all four repros crash (exit 101) again, so the guard is not vacuous. Also folds in two verification guards hardened during the same hunt -- vafautodiff (8->16 checks: cross-derivatives with both arguments live, the blind spot that hid E-185's hypot bug, plus a 44-point battery) and operator_examples (+14 const-vs-runtime checks: every check there used literal operands, exercising only the folder -- the side E-37's >> bug happened to be on). New examples/arraycast_examples (23 checks). Regression 174/174.

E-215 *hir_ty (builtin.rs), hir_lower (callbacks.rs, expr.rs), osdi (compilation_unit.rs, stdlib.c), examples/plusargs_examples/**

$test$plusargs / $value$plusargs productionized. E-12 had gated these as mechanism-unavailable fallbacks ($test always FALSE, $value never wrote its output). E-215 serves them through the simparam channel (ngspice publishes each command-line +name[=value] as namespaced simparams -- see the ngspice report). COMPILER side: $test$plusargs("name") lowers to simparam("$test$plusargs$name",0)!=0; $value$plusargs("name=%fmt",var) builds the key from the compile-time literal and reads by TARGET TYPE -- $valnum for int/real (ficast for an int target), and a NEW non-fatal simparam_str_opt for a string target. simparam_str (E-25) FATALS on an unknown name, which a plusarg probe must not, so simparam_str_opt (stdlib.c + the SimParamStrOpt callback + compilation_unit wiring) returns the default instead. hir_ty: VALUE_PLUSARGS's 2nd arg became an OUTPUT target -- three signatures Var(Integer)/Var(Real)/Var(String) (mirroring $random's seed / $fgets's buffer) instead of the read-only Val(String) that made extraction impossible; TEST_PLUSARGS's arg is now Literal(String). DESIGN NOTE: reading the value through the op-dependent simparam channels DELIBERATELY avoids the $sscanf scanner, whose scanf_begin/scan_* thread a hidden module-global cursor with no explicit data dependency -- the setup/eval partitioner can hoist a bias-independent scan_* into instance setup while its scanf_begin stays in eval, so the scan reads a NULL cursor and segfaults (observed, then designed out). $value$plusargs matches only the name=value form per the LRM. No OSDI ABI change; old .osdi unaffected. Verify (examples/plusargs, 12 checks both solvers). Regression 175/175 (new example folder).

E-219 *preprocessor (grammar.rs), basedb (diagnostics/sink.rs), examples/robustness_examples/**

preprocessor macro-argument hang + diagnostic-flood cap. Re-running the openvaf-r robustness campaign against the shipped binary (see the robustness report) found a FIFTH hang path the E-147/E-148 guards did not cover, at a ~2% mutation-fuzz rate. FINDING A -- INFINITE LOOP: a backtick token followed by `((name() is a macro call, and the preprocessor collects its parenthesised argument list token by token in parse_macro_call -> parse_macro_token. The SAME non-advancing pattern exists in the define PARAMETER list loop (parse_define). Neither collector had a forward-progress guarantee: when a non-Macro compiler directive (`include, ifdef, endif, undef, ...`) appeared inside the argument list, parse_macro_token pushed an UnexpectedToken error and RETURNED WITHOUT CONSUMING the token, so the caller's collection loop re-examined the same directive token forever -- an unbounded diagnostics-vector growth that pins the CPU (a `sample` of a hung compile named the loop exactly: process_token -> parse_macro_call -> parse_macro_token -> RawVec::grow_one -> realloc). It is trivially reached: injecting `(anywhere near an include (or any macro token) splits the directive so the rest of the file is scanned as a macro argument. (define did NOT trigger it -- compiler_directive() has no define arm, so it falls through to Macro and is consumed as a bogus macro name, which happens to advance; only the RECOGNISED directives hit the non-advancing branch.) A stray delimiter in a define parameter list that is neither an identifier,) nor , (e.g. a / or " in a corrupted define) is likewise consumed by none of the loop's expect/eat calls, so parse_define spins on it. FIX (grammar.rs): guarantee forward progress -- the stray-directive branch of parse_macro_token consumes the offending token after reporting it (p.bump); and BOTH collection loops (macro-call args and define params) gained a backstop that records the source offset and bails with a clean UnexpectedEof if an iteration consumes nothing, so no non-advancing token, present or future, can spin them. No valid model puts a directive inside a macro call's parentheses, so valid input is unaffected. FINDING B -- DIAGNOSTIC FLOOD: with A fixed, deeply nested NON-macro garbage (e.g. sin(x3000 in a declaration) no longer looped but still took ~40s -- it produces thousands of parse errors and codespan_reporting builds a full source-annotated report for EVERY one; rendering ~3000 diagnostics (each extracting and laying out its surrounding source) is the entire cost (it persists with stderr redirected to /dev/null -- it is the BUILDING, not the writing). A bounded-but-40-second rejection is still a DoS vector. FIX (sink.rs): cap the number of diagnostics the console sink RENDERS at 128; beyond that only the counters advance and a one-line 'further diagnostics suppressed' note prints. summary() still reports the true error total and the exit code is unchanged -- the standard 'too many errors' behaviour of rustc/clang, which does not affect any input with <=128 diagnostics (every real model and every ordinary mistake). RESULT: name(+ stray include: infinite -> clean error <0.01s; real model + injected (': >90s -> <0.05s; sin(x3000 in a declaration: ~40s -> <1s; a few real errors: unchanged, all shown. Continuation of E-148 (E-148 hardened the parser expr-depth / include recursion / array expansion; E-219 covers the two paths E-148 did not touch -- the preprocessor macro-arg collector and the diagnostic sink). Verify: examples/robustness_examples gains eight argument-collection cases -- five macro-call (m(then include/ifdef/endif/undef, and one with 4000 leading '(') and three define parameter-list (M(a,/,b), M(a"b), M(a;b)) -- plus a valid macro-call-with-nested-parens regression (26/26); the production corpus (VA_TEST/compile_all.py) still compiles 92/92 standalone models to the identical verdict; the campaign fuzzer's ~2% hang rate (3000 iterations) drops to 0 hangs, 0 crashes. Two files (openvaf/preprocessor/src/grammar.rs, openvaf/basedb/src/diagnostics/sink.rs). No change to any valid program's output beyond the >128-diagnostic suppression note.

E-220 *parser (parser.rs), syntax (lib.rs), preprocessor (grammar.rs, sourcemap.rs), basedb (diagnostics.rs), hir_ty (diagnostics.rs, inference.rs, validation.rs, validation/body.rs), examples/vafcrash2_examples/**

crash hardening round 2: ten compiler panics -> clean errors. A second robustness pass, continuing E-213 and E-219. Re-running the mutation fuzzer against the shipped compiler with DIVERSE compact-model seeds (BSIM/HICUM/PSP/HiSIM/VBIC/MEXTRAM/EKV) and new mutation strategies (keyword, attribute (* *), and bracket injection alongside byte/truncate/delimiter/deep-nest) found ~5% of mutated inputs still CRASHED the compiler (exit 101, 'OpenVAF encountered a problem and has crashed!') rather than reporting an error. Every one is the E-213 pattern -- a panic/assert/unwrap/index a valid model never reaches but malformed input does (all caught by the E-213 panic hook, so never memory-unsafe; the bug is the crash UX + missing diagnostic). TEN root causes, all fixed: (1) parser/parser.rs -- the parser can spin in error recovery; a step counter assert!(steps<=10M,'seems stuck') panicked. It now signals EOF at the limit so parsing winds down and reports its errors. (2) basedb/diagnostics.rs -- a diagnostic built from an empty node list hit to_unified_span_list([])=>unimplemented!(); it renders label-less, anchored to the new SourceMap::root_file(). (3) preprocessor/grammar.rs -- TextRange::new(start, end) with start>end (a macro-argument/`define`/include span at EOF) tripped a text-size assert; five sites clamp end>=start. (4) hir_ty/diagnostics.rs + validation.rs -- 28 sites did expr_map_back[e].unwrap()/stmt_map_back[s].unwrap(); a SYNTHESIZED expression/statement has no source-map-back entry, so reporting a type error on it panicked. They resolve the span through a fallback (empty range) via one expr_range helper. (5) hir_ty/validation/body.rs -- 11 sites did expr_types[arg].unwrap_node()/unwrap_branch()/unwrap_port_flow(), which unreachable!() when a builtin (e.g. $port_connected) or nature access gets a wrong-typed argument inference did not reject; they bail cleanly (match {Ty::X(id)=>id,_=>return}). (6) syntax/lib.rs -- to_ctx_span mapping a span across source contexts could yield start>end in TextRange::new; clamp. (7) preprocessor/sourcemap.rs -- FileSpan::with_subrange asserted the subrange fit its parent (E-213 fixed one trigger at its source; this makes the mapping itself total); clamp into the parent. (8) preprocessor/grammar.rs -- stripping include quotes with path[1..len-1] panicked on a malformed/unterminated string literal (a lone quote); saturating_sub + get make it total. (9) hir_ty/inference.rs -- resolving a call whose arguments match NO overload left the candidate list empty after the semantic-retain and candidates[0] indexed out of bounds; it falls back to the pre-filter set (mirroring the existing restore after the exact-match retain). (10) hir_ty/validation/body.rs -- the builtin-validation arms index args[0..2] assuming the call has as many arguments as the builtin requires; a call with too few (e.g. $simparam(), $port_connected()) indexed out of bounds. ONE entry guard skips builtin validation when args.len() < BuiltinInfo::from(call).min_args -- inference already reports the ArgCntMismatch. Method: the fuzzer classifies OK/clean-error/CRASH/HANG; each crash was triaged from its crash log to the innermost openvaf frame, grouped, and the panic read at the source; fixes were applied one cause at a time, each verified against the recorded crash corpus, then a fresh fuzz re-checked convergence (each pass exposed the next-rarest site -- a classic fuzzing tail -- until the rate hit zero). Result: the ~390-input recorded crash corpus all clean-errors; a fresh 12000-iteration fuzz on the fixed compiler is 0 crashes / 0 hangs (down from ~5%); the 92/92 standalone production models compile to the identical verdict; parser/syntax/preprocessor/basedb/hir_ty/hir/sim_back unit suites pass. Verify (examples/vafcrash2): 19 checks with hand-crafted inputs targeting each cause, MUTATION-TESTED (reverting the fixes makes the guarded inputs crash again, so the suite is not vacuous). No OSDI/ABI change; generated .osdi for every existing model is identical. Eight files. Regression 179/179 (new example folder).

E-230 *hir_def (item_tree/lower.rs), hir_ty (validation.rs), syntax (ast/expr_ext.rs), examples/vafcrash3_examples/**

crash hardening round 3: three compiler panics -> clean errors. A third robustness pass (following E-213/E-219/E-220). The production corpus recompiled (92/92 standalone, identical verdicts) plus a fresh ~19,500-iteration mutation-fuzz over diverse compact-model seeds found THREE more distinct panic root causes, all the E-213 class (caught by the panic hook, memory-safe, but a crash instead of a diagnostic). (1) hir_def/item_tree/lower.rs -- a `begin :` sequential block with the scope colon but a MISSING/invalid name identifier has `block_scope().is_some()` yet

‘name = block_scope().and_then(

E-261 *mir_autodiff (builder.rs), examples/vafsqrtguard_examples/**

autodiff: guard the **`sqrt()`** derivative singularity. `d/dx sqrt(x) = 1/(2*sqrt(x))` is `+inf` at ngspice's default `x=0` DC initial guess; openvaf-r emitted that raw `+inf` into the Jacobian, which produced a `nan` and made the operating point fail outright -- a conductor `I=K*sqrt(V)` never found ANY DC op (dynamic/true gmin, source, and pseudo-transient stepping all died, node = `nan`). The tell was an internal inconsistency: `Pow` carried an explicit `base==0 -> derivative:=0` guard ”to ensure numerical stability” while `Sqrt` had none, so the mathematically identical `pow(V,0.5)` and `V**0.5` converged and ngspice's own behavioral B-source `sqrt(v(n))` converged, but `sqrt(V)` NaN-failed. (The `Pow` guard is itself incomplete -- a block split that only protects a bare terminal `pow`/`sqrt`; `2.0*pow(V,0.5)` fails on the unmodified compiler because the downstream multiply consumes the raw derivative.) FIX (`inst_cache`): change the `Sqrt` derivative cache from `2*sqrt(x)` to `2*sqrt(x + a)` (`a = 1e-18`) -- the exact derivative of the smoothly regularized `sqrt(x+a)`, so the emitted derivative is `x'/(2*sqrt(x+a))`. It is FINITE at `x=0` (`1/(2*sqrt(a)) ~ 5e8`, a large but bounded conductance -> small controlled Newton steps that creep out of the singularity, exactly like the B-source); EXACT for `x>0` (the nudge is INSIDE the root, so the perturbation is `~a/(2x)`, below the ULP -- no finite-bias derivative changes, higher-order derivatives and the internal `sqrt(1-x^2)` of asin/acos/asinh/acosh/atanh included); and COMPOSABLE (being a plain value it propagates through downstream operators `K*sqrt(x)`, `1/(1+sqrt(x))`, `exp(-sqrt(x))`, unlike the block-split guard). Two alternatives were rejected: a block-split `Sqrt` arm mirroring `Pow` (does not compose -- scaled `sqrt` still fails), and an additive `2*sqrt(x)+delta` (perturbs the derivative `~delta/(2*sqrt(x)) ~ 1e-9`, breaking the bit-exact higher-order asin/acos autodiff unit tests) -- the `sqrt(x+a)` form is exact to the ULP AND convergent, so NO test tolerance was loosened. `pow(x,fractional)` is unchanged (its base derivative `(y/x)*x^y` is an `inf*0` form in the shared `Pow`/`Atan2` chain rule with no clean branchless regularization; `sqrt()` is the standard spelling). Verify (examples/vafsqrtguard, both solvers): bare and strongly-scaled `K*sqrt(V)` (K=1,2,5) find their true KCL op -- not `nan` -- and match the equivalent B-source to ~1e-5; the guarded derivative `K/(2*sqrt(V))` is exact for `V>0` (~1e-6); composed `G0/(1+sqrt(V))` converges; `pow(V,0.5)` now agrees with `sqrt(V)`. openvaf-r's own autodiff suite (incl. numeric third-order asin/acos exactness at `10*eps`) and the OSDI/sim_back MIR snapshots pass unchanged in value. Two source files.

E-262 *mir_autodiff (builder.rs), examples/vafsqrtguard_examples/**

autodiff: guard the **`pow(x,y)`** base-derivative singularity (the E-261 sqrt fix, applied to pow). For `0<y<1`, `d/dx pow(x,y) = y*x^(y-1)` is `+inf` at the `x=0` DC initial guess -- the same singularity as sqrt (pow(x,0.5) IS sqrt(x)). openvaf-r builds it via the chain rule shared by Pow and Atan2: `d(pow) = (x'*cache[0] + y'*cache[1])*cache[2]` with `cache[0]=y/x`, `cache[2]=x^y`, so the base term `x'*(y/x)*x^y` is `+inf*0 = NaN` at `x=0` (the `y/x` factor is `+inf`, the `x^y` factor is `0`) -- it NaN-poisons the Jacobian and the operating point fails. Pow carried a PARTIAL guard (a block split forcing the derivative to 0 when `base==0`), which is why a BARE terminal `pow(V,0.5)` converged -- but a block split cannot compose: a downstream instruction (`2*pow(V,0.5)`, `1/(1+pow(V,0.5))`) consumes the RAW derivative computed inside the conditional block, so scaled/combined fractional pow still NaN-failed on the unmodified compiler. FIX (`inst_cache`, mirroring E-261's sqrt): cache the derivative of the a-shifted `pow(x+a, y)` (`a = 1e-18`) while the VALUE `res = x^y` is unchanged -- `cache[0] = y/(x+a)`, `cache[1] = ln(x+a)`, `cache[2] = (x+a)^y`. The shared chain rule then yields base term `x'*y*(x+a)^(y-1)` and exponent term `y'*ln(x+a)*(x+a)^y`, both FINITE at `x=0` (`y*a^(y-1)` is large but bounded -> a controlled Newton step out of the singularity, like the sqrt guard and ngspice's B-source), EXACT for `x>0` (the nudge is inside the power, perturbation `~a/x` below the ULP), and PLAIN VALUES so they compose through downstream operators. With a composing branchless guard in place the old block-split Pow guard is removed. Atan2 -- which shares the chain-rule ARM but has its own cache -- is untouched and verified unchanged; `y>=1` is a no-op (no singularity); the solution-dependent exponent term is now

finite at x=0 too (ln(x+a) not ln(0)=−inf). ln(x)/1/x are left as-is (their VALUE, not just the derivative, is +-inf at 0, so no derivative guard well-poses a model that evaluates them at 0). Verify (examples/vafsqrtguard [6]/[7], both solvers): bare and strongly-scaled K∗pow(V,Y) (K=1,2,5 at Y=0.5,0.3,0.25) find their true KCL op -- not nan -- where the unmodified compiler NaN-failed the scaled/fractional cases; the guarded derivative K∗Y∗V^(Y−1) is exact for V>0 (~1e-6). openvaf-r's autodiff suite (incl. atan2 and higher-order checks) and the OSDI/sim_back MIR snapshots pass unchanged in value; the many production models using pow (BSIM/PSP/...) are bit-unchanged for V>0. Completes the autodiff singular-derivative work (E-261 sqrt + E-262 pow). Two source files.

E-263 *sim_back (init.rs), hir_ty (inference.rs), hir (elaborate.rs), test_data/ui/ddx.log, examples/vafcrash4_examples/**

robustness fuzzing: three compiler panics -> clean errors (the 4th such pass, following E-213/E-219/E-220/E-230). Three fuzzing strategies against the committed compiler -- byte/token mutation of the whole .va corpus, grammar-aware STRUCTURED adversarial inputs, and VALID-but-pathological modules that compile through to the backend -- surfaced one distinct panic each (all the E-213 class: panic-hook-caught, memory-safe, but a crash instead of a diagnostic). (1) sim_back/init.rs -- deeply nested analog operators (I<+absdelay(ddt(ddt(absdelay(..))))) produced a value tagged for the instance-setup cache whose mapping in the INIT function is a Const or has no definition at all (ValueDef::Invalid); build_init_cache assumed every cached value is a computed instruction result and called unwrap_result()/unwrap_inst(), and even once survived the OSDI LLVM backend read a BuilderVal::Undef. FIX: before allocating a cache slot, handle the non-instruction cases -- substitute a Float/Int/Str/Bool CONSTANT init value directly into the eval function (eval uses the init-time value; no runtime slot needed) and treat an Invalid mapping like the existing dead-value path (default to 0). Only a genuine instruction result takes the slot+optbarrier path, so no cache slot or codegen ever sees a constant/undefined value again. (2) hir_ty/inference.rs -- ddx(V(p,n), 5) (a ddx whose 2nd arg is not a potential/flow probe) crashed hir_lower at value_def(unknown).unwrap_param(). The type-checker HAD an "invalid ddx unknown" diagnostic, but its guard tested expr (the ddx call itself, ALWAYS an Expr::Call) instead of unknown, so the diagnostic was DEAD CODE and the malformed call slipped through to codegen. FIX: test unknown -- a non-call unknown (a literal, a plain variable) now raises the diagnostic and compilation stops before lowering. (This also newly (correctly) diagnoses the ddx(1.0,1.0)/ddx(1.0,foo) "random fuzz" cases the ui/ddx.va test already carried but that the dead code never rejected -- ddx.log updated to 8 errors.) (3) hir/elaborate.rs -- a malformed top-level module whose item TREE recorded an instantiation but whose parsed AST module_items() came back empty (parser error-recovery and item-tree construction disagreeing) crashed the E-5/E-49 hierarchy-flattening pass at items.first().unwrap(). FIX: an empty AST item list means nothing to flatten -- return the module text verbatim, the same no-op the pass already takes for a module with no instantiations. Behaviour-preserving for valid input: full dual-solver regression 214/214, cargo test unchanged except the intended ddx.log update (no MIR/OSDI snapshot changed -- no real model exercises the new paths), and a re-fuzz of the fixed compiler across all three strategies finds no surviving panics. Verify (examples/vafcrash4): a minimal reproducer per cause now compiles cleanly (nested analog ops) or clean-errors (ddx, malformed module), each having exited 101 on the shipped binary. Three source files + one snapshot.

E-264 *hir (elaborate.rs), osdi (lib.rs), examples/vafhang_examples/**

large instance arrays: hierarchy-flatten O(N²)->O(N) + OSDI-codegen stack headroom for deep per-node fan-in. Two scalability/robustness defects on the same path -- compiling a module that instantiates a large array of a sub-module. (1) FLATTEN O(N²)->O(N) (hir/elaborate.rs): the E-5/E-49/E-86 pass renders one textual copy of a sub-module per instance with a per-instance name prefix and rewrites hierarchical name references, running N times for leaf u[0:N]. Three inner steps were themselves O(N), making it O(N²) -- doubling N QUADRUPLED time (2k≈1.

8s, 8k≈30s, 16k≈100s), so a big array looked like a hang: hierarchical-name resolution (`find_instance_path_holes`) re-scanned every instance prefix per token via `.keys().any(..)`; it was called per port binding, per instance (multiplying by N again); and each per-instance scope `clone()`d the whole E-86 absolute-reference map (O(N) entries). FIX (behaviour-preserving): precompute an ANCESTOR SET once for O(1) "is this chain a hierarchical prefix?" tests (no per-token scan); a DOT-FREE EARLY-OUT (every hierarchical ref contains a `.`, so the common no-dot token skips resolution entirely); and share the absolute-reference map by reference -- a small `Rc<AbsPrefixes>` (map + its precomputed ancestor set) is `Rc::cloned` (O(1)) into each scope instead of deep-copied. Now O(N): 16 001 instances elaborate+compile in ~1s (was ~100s), 32 001 in ~2s; the elaborated text and every resulting `.osdi` are IDENTICAL (only faster). (2) CODEGEN STACK (`osdi/lib.rs`): when many instances contribute to the SAME node and don't collapse (distinct per-instance parameter), the residual is a chain of thousands of accumulated contributions that OpenVAF's recursive OSDI codegen + autodiff walk; that recursion ran on a rayon codegen worker whose default stack is only a few MB, so past ~8 000 contributions it aborted with `thread ... has overflowed its stack` / SIGABRT -- a crash on large-but-valid input. FIX: run OSDI codegen on a dedicated rayon pool built with a 256 MiB worker stack instead of the default global pool. A thread's stack size is a property of the thread, not the work -- it cannot change the emitted code (the `.osdi` is byte-equivalent modulo the linker's already-nondeterministic build id) -- it only lets the deep recursion complete; the 8 001-contribution design now compiles cleanly. Behaviour-preserving: full dual-solver regression passes and `cargo test` is unchanged (no MIR/OSDI snapshot moved -- the flatten fix only accelerates identical work; the stack fix is a thread property). Extreme instance counts remain bounded by the existing 1 M array/generate caps and downstream compile cost. Verify (examples/vafhang): check A (always) -- 16 001- and 32 001-instance arrays compile in ~1s/~2s within generous absolute bounds a re-introduced O(N^2) would blow; check B (`--slow`) -- the 8 001-contribution deep-fan-in design compiles cleanly (~38s) where the pre-fix binary SIGABRT'd. Two source files.

E-265 *hir_ty (inference.rs), examples/vaflaplace_examples/**

`laplace_*`/`zi_*` coefficient argument: compiler panic -> clean diagnostic (fifth robustness-campaign find, following E-213/-220/-230/-263). A num/den (pole/zero) argument must be a real coefficient array (LRM 9.19). `laplace_*` bypasses the generic argument checker (which would reject a bare array-variable reference before the operator's special case can accept it) and uses `infere_array_arg`, whose fallback returned the inferred type WITHOUT requiring a real value -- so a bare NET reference (`laplace_nd(1.0,1.0,p)`), a branch, or a string was typed but never rejected, reached `hir_lower`, and panicked (`resolve_path`: "invalid HIR: path .. was not resolved", exit 101) when the coefficient elements were lowered as values. Every ordinary value context (and the laplace INPUT arg) already rejects a net-as-value cleanly; only the coefficient arg was different. FIX: the check goes in `infere_laplace`, NOT in the shared `infere_array_arg` -- that helper is also used by E-33 `case` discriminants/items and E-34 concatenations, which legitimately carry integer/string values, and forcing real there made integer `case` PANIC (caught in regression before shipping). `infere_laplace` now requires each num/den type to be a real/integer scalar or array (`Ty::to_value()` is `None` for a net/branch reference -> rejected), emitting the normal "expected real value but found .." type-mismatch; the array-literal / array-variable / scalar shapes are unchanged. A second adjacent crash from the same fuzzing -- an empty DIRECT denominator (`laplace_nd(V,1.0,'{})`, where the state-space realization computes `den.len()-1` and reads `den[n]`, underflowing/OOB) -- is rejected too (the operator kind is now passed to `infere_laplace` to tell a direct denominator from a `*_np`/`*_zp` pole list); an empty NUMERATOR (H=0) and an empty POLE list (denominator polynomial 1) stay legal. Behaviour-preserving: full dual-solver regression passes (incl. the laplace/zi filter models in the complex-pole and RF-convolution suites), `cargo test` UI snapshots unchanged, and a re-fuzz of 6000 random malformed `laplace_*`/`zi_*` calls finds no surviving panic. Verify (examples/vaflaplace, 15 checks): six malformed coefficient args (net-den/net-num/branch/string/net-in-zi_zp-roots/empty-direct-den) now clean-error; well-formed shapes (real/int array literals, scalar, array-var

ref, zi_nd, empty numerator, empty pole list) still compile. One source file.

Enhancements not listed (57, 60, 62–64, 69, 72–77, 79–83, 94–95, 98–100) changed no compiler sources — they were validation suites, documentation, benchmark tooling, or ngspice-side work (see the ngspice report).

E-286 *mir_opt (const_eval.rs, simplify.rs), examples/vafcodegen_examples/**

constant-folding an integer division by zero killed the compiler. `q = 5 / 0;` exited openvaf-r with an internal error and produced no `.osdi`, while a *runtime* zero divisor had always been accepted (the generated code just performs the division) -- an inconsistency, not a policy. CAUSE: `eval_binary`'s integer arm evaluated every opcode directly (`func.dfg.iconst(lhs / rhs)`), so with `rhs == 0` the division happens INSIDE the compiler process and the compiler is what dies; `i32::MIN / -1` overflows the same way. The neighbouring arms had a quieter version of the same defect -- `Iadd/Isub/Imul` folded with checked arithmetic and the shifts folded by an unconstrained distance, so `2147483647 + 1` and `1 << 40` were folded in a way that does NOT match what the generated code computes (LLVM emits plain two's-complement wrapping arithmetic; a shift distance outside `0..32` is poison). This is the `const_eval == codegen` invariant: the folder must produce exactly what the runtime path would, or decline. FIX: `eval_binary` returns `Option<Value>` (the convention `eval_unary` already used) -- it declines for div/rem by zero, `i32::MIN/-1`, and shift distances outside `0..32`, leaving the instruction on exactly the runtime path a non-constant divisor takes; and folds add/sub/mul with `wrapping_*`. The single call site in `simplify.rs` forwards the `Option`. Verify (examples/vafcodegen): `constfold.va` exercises all five shapes in one module and compiles, where the pre-fix compiler exited 101. `cargo test` unchanged (no MIR/OSDI snapshot moved); full dual-solver regression passes. Two source files.

E-287 *mir_opt (simplify_cfg.rs), examples/vafcodegen_examples/**

a const-folded branch orphaned a block, leaving a stale phi edge (broken SSA). A noise operator in an `if` CONDITION is zero outside noise analysis, so the optimizer proves the branch constant and folds it -- producing a function that violates SSA. CAUSE: `const_fold_terminator` rewrites the branch to a jump and calls `remove_phi_edges(dead_dst, bb)`, which fixes the phis INSIDE the newly-unreachable successor -- but the phis that actually go stale are the ones in ITS successors, which keep an edge labelled with the orphaned block, naming a value only ever available through the edge just deleted (`v22 = phi [v31, block2], [v20, block3]` where block2 is now unreachable and v31 is defined in a block that reaches block3, not block2). `simplify_bb` IS the pass that collects predecessor-less blocks and prunes their successors' phi edges -- but only on a LATER sweep, and blocks are visited in layout order so the orphan is typically already behind the cursor. The sweep never ran again because this branch of `const_fold_terminator`, unlike its `then_dst == else_dst` sibling three lines above, never set `local_changed`; with no change flagged, `iteratively_simplify_cfg` stopped with the orphan in place. Because the MIR verifier is a `debug_assert!`, the release compiler carried the malformed function forward silently. FIX: set `local_changed` after folding, so the driver sweeps again -- monotone (a folded branch becomes a jump and cannot fold again), so termination is unaffected. Verify (examples/vafcodegen): `orphanblock.va` compiles and simulates (`I == V/1k`); an assertions-enabled compiler now accepts it where it reported `v31 doesn't dominate use (block11 !dom block2)`. This pass runs on EVERY model, so the whole 248-model example corpus was recompiled release-vs-release: zero regressions. One source file, one statement.

E-288 *mir_llvm (intrinsics.rs), examples/vafcodegen_examples/**

`hypot` was declared with one parameter and called with two. `I(a,b) <+ hypot(V(a), V(b))` produced a module LLVM rejects ("Incorrect number of arguments passed to called function!"). CAUSE: `hypot` needs a special case in the intrinsics table (Windows spells it `_hypot`), and that case declared `&[t_f64]` while `builder.rs` emits a two-argument call. Every other binary entry -- `atan2` and `llvm.pow.f64`, both a few lines away -- is correct; `hypot` is the odd one out because

it sits outside the `ifn!` macro block. Constant arguments fold before codegen, so only a runtime argument reached the bad declaration. It survived because the check that reports it (`llmod.verify_and_print()`) is a `debug_assert!` -- release builds never run the module verifier. FIX: declare it `&[t_f64, t_f64]`, matching the call and its neighbour `atan2`. Stated plainly: on arm64 macOS the malformed call still produced the RIGHT number (the extra argument lands in the register the callee reads anyway) -- this was invalid IR LLVM is licensed to miscompile under a different target, calling convention, or optimization pipeline, not a demonstrated wrong answer on this platform. Verify (examples/vafcodegen): `hypotclog2.va` uses runtime parameters and checks `hypot(3,4) == 5` exactly. One source file, one argument list.

E-289 *mir_llvm (intrinsics.rs, builder.rs), examples/vafcodegen_examples/**

`llvm.ctlz` was declared without its type suffix. `\$clog2(n)` with a runtime argument produced invalid IR (”Intrinsic name not mangled correctly for type arguments! Should be: llvm.ctlz.i32”). CAUSE: `llvm.ctlz` is an OVERLOADED LLVM intrinsic -- its name must carry the type it operates on -- but it was registered and looked up under the bare name. The neighbouring overloaded entries are all spelled correctly (`llvm.pow.f64`, `llvm.sqrt.f64`, `llvm.ceil.f64`, `llvm.lround.i32.f64`); `ctlz` is the only one missing its suffix. It backs `\$clog2` (which computes `bit_width(n-1)`), so every model calling `\$clog2` on a non-constant argument emitted invalid IR; as with E-288 the module verifier that reports it is a `debug_assert!`, so release builds shipped it. FOUND BY replaying the committed example corpus through an assertions-enabled compiler: `clog2_examples/clog2_demo.va` -- a model that had been shipping and simulating correctly -- was rejected. FIX: register and look it up as `llvm.ctlz.i32` (both files, including the ”intrinsic not found” message). Verify (examples/vafcodegen): `hypotclog2.va` checks `\$clog2(100) == 7` with a runtime parameter; the pre-existing `clog2_examples` suite continues to pass and now also passes under an assertions-enabled compiler. Two source files.

E-290 *osdi (inst_data.rs), examples/vafcodegen_examples/**

`\$temperature` as an operator argument used the wrong struct-GEP type -- the shipped compiler SIGSEGV'd. `I(out) <+ ac_stim("ac", \$temperature, 0.0)` killed openvaf-r with exit 139 while optimizing the model, behind the malformed IR `getelementptr inbounds double, ptr %0, i32 0, i32 5` (”Invalid indices for GEP pointer type!”). CAUSE: `LLVMBuildStructGEP2`'s first argument is the AGGREGATE being indexed -- the instance-data struct -- and the `ParamKind::Temperature` arm passed the FIELD type (`cx.ty_double()`) instead. A two-index GEP is only meaningful on an aggregate, so the IR is invalid; and the offset it describes is a flat `5 * sizeof(double)` = 40 bytes rather than `offsetof(instance, temperature)`. `TEMPERATURE` is field 5 and fields 0..4 are the param-given bitfield, the two Jacobian pointer arrays, the node mapping and the collapse flags -- variable-length arrays whose combined size is essentially never 40 bytes -- so even where LLVM did not crash the load landed on unrelated bytes. Every sibling gets this right (`eval_output_slot_ptr`, `temperature_loc` both pass `self.ty`). The bug was in TWO places: `load_eval_output` (the noise / `ac_stim` argument path) and `nth_opvar_ptr` (the operating-point-variable read path). Only `\$temperature` read DIRECTLY as an operator argument takes this path -- a computed variable (`tk = \$temperature;`) lowers to an eval-output slot and was always correct, which is why ordinary models never tripped it. As with E-288/-289 the module verifier is a `debug_assert!`, so a release build had nothing to stop it before LLVM's optimizer hit the malformed GEP. FIX: pass `inst_data.ty` at both sites. Verify (examples/vafcodegen): `tempacstim.va` compiles (pre-fix: exit 139) and through a 1 ohm load reads back the nominal 300.15 K. One source file, two call sites.

E-291 *hir_lower (stmt.rs), examples/vafcodegen_examples/**

`max`/**`min`**/**`abs`** in a **`case`** default arm left a block unsealed. `case (V(a,b)) 5.0: y = 11.0; default: y = max(3.0, 7.0); endcase` aborted the compile with ”FunctionBuilder finalized, but block N is not sealed”. The discriminator is sharp: `pow(2,3)` in the same arm is fine, `max` in an ITEM arm is fine, `max` outside a `case` is fine -- only a branch-lowering builtin in the DEFAULT

arm failed. CAUSE: `max/min/abs` do not lower to a single instruction; they lower through `make_cond` to a real select with its own then/else/merge blocks (`pow` and friends emit one instruction and open no blocks). `lower_case` creates a fall-through block per case item and switches to it, but leaves it to be sealed by an `ensured_sealed()` -- at the top of the next iteration, or, for the LAST item, after the default arm's body is lowered. `ensured_sealed()` seals whatever block the builder is currently positioned in; when the default arm's body opens blocks of its own it leaves the builder on ITS merge block, so the seal lands there (already sealed, a no-op) and the case's fall-through block is never sealed at all. FIX: seal it where it is created -- the branch just emitted is its only predecessor, so all predecessors are known and immediate sealing is correct; the later `ensured_sealed()` calls check `is_sealed` first and become no-ops. Verify (examples/vafcodegen): `casemax.va` compiles AND still picks the right arm (`V=2` -> default `max(3,7)=7`, `V=5` -> item arm `11`); also checked for `min`, `abs`, nested combinations, an integer discriminant, and `casex`. One source file, one statement.

E-292 *sim_back (topology/small_signal_network.rs), examples/vafcodegen_examples/**

small-signal pruning indexed a key its own replay never inserted. A fuzzer input routing noise waves through `idt` into nested `laplace_nd` coefficient arrays aborted the compile with "no entry found for key". CAUSE: `prune_small_signal` moves a linear contribution into its own dimension "where possible" (its own doc comment) -- but whether it IS possible is decided by two different pieces of code: `collect_linear_contributes` classifies the contribution as linear, while the replay inside `create_dimension` is what actually BUILDS the per-dimension value and records it in `val_map`. The replay deliberately declines several shapes (an `fmul` whose BOTH operands depend on the dimension; any opcode falling through to its catch-all), so the two analyses can disagree -- and when they did, `self.val_map[&contribute]` panicked. FIX: treat the disagreement as what it is -- pruning is a best-effort optimization -- and give up on that value instead of crashing. The bail-out path must ALSO run `replace_uses(placeholder, val)`: `prune_small_signal` creates an invalid PLACEHOLDER value before the replay and resolves it at the end, so an early `continue` that skipped it would leave an invalid value in the function. The replay instructions that go unused are dead code the later DCE pass removes. Verify (examples/vafcodegen): `ssprune.va` (the reduced reproducer) compiles where the pre-fix compiler exited 101. Behaviour-preserving wherever the two analyses agree, which is every real model: the full 248-model example corpus compiles release-vs-release with zero regressions, including the noise and RF suites that exercise this pass most heavily. One source file.

E-293 *sim_back (topology/lineralize.rs), examples/vafcodegen_examples/**

one analog operator nested directly inside another. `I(a,b) <+ ddt(ddt(V(a,b)))` crashed the compile -- as did three deep, or split across a variable (`x = ddt(V); I <+ ddt(x);`) -- while putting anything at all in between (`ddt(2.0*ddt(V))`) always worked. CAUSE: `builid_analog_operators` materializes each classified operator; an `Evaluation::Equation` allocates an implicit unknown, calls `replace_uses(res, eq_val)` and then DELETES the operator's instruction, while an `Evaluation::Linear` adds a stored `dimension` value into the contribution's reactive part. Those dimension values live in the `Evaluation::Linear { contributes }` triples -- OUTSIDE the data-flow graph -- and `replace_uses` only rewrites operands held inside the DFG, so it cannot reach them. With direct nesting the stored dimension IS the inner operator's result, so once the inner operator is processed the outer one holds a value whose defining instruction has been removed (`LINEAR inst1 uses dimension=v17 (def = Result(inst0,0))` where inst0 was just deleted); with an `fmul` in between the replay yields a fresh value (`Result(inst2)`, the multiply) and the situation never arose. Everything derived from the dangling value surfaced later as `invalid argument vN` when the instance-init function was validated. FIX: iterate the operator list by index so an operator can fix up the entries still pending behind it, and retarget any pending dimension naming this operator's result -- `eq_val`, the implicit unknown the inner operator became, is exactly what the outer operator's reactive contribution should use, so this states the correct second-derivative formulation rather than merely removing a dangling reference. The

Evaluation::Dead arm carries the same hazard and is retargeted to F_ZERO; the reverse order is already safe (a Linear processed first inserts a genuine DFG use, which a later Equation's replace_uses does reach). Verify (examples/vafcodegen): in AC a ddt is j*omega, so a second derivative is (j*omega)^2 = -omega^2 -- ‘

E-294 *mir_opt (simplify_cfg.rs, dead_code_aggressive.rs), examples/vafcodegen_examples/**

a **Branch->Jump** rewrite left the condition in the use list. A Branch carries exactly one value operand (its condition); a Jump carries none, so every such rewrite must retire that operand's use-list entry. Two of the four sites simply overwrote the instruction (insts[terminator] = Jump{..}), leaving the record linked into the condition value's list naming OPERAND 0 of an instruction that now has ZERO operands -- use_to_value then indexes an empty slice (”index out of bounds: the len is 0 but the index is 0”). Offenders: simplify_bb's empty-exit-block rewrite (both arms), and dead_code_aggressive's dead-block terminator rewrite (same defect, found by inspecting the class rather than from a failing model). The two rewrites in const_fold_terminator do it correctly -- one via zap_inst, one via detach_operand -- which is what made the omission legible as an inconsistency. REACHED BY a narrow shape: \$fatal exits the analog block so its arm becomes an EMPTY EXIT block, exactly what simplify_bb rewrites, and the condition must be a PARAMETER compare (with a node voltage the arm is not an empty exit block and the rewrite never fires; \$finish does not produce this shape either) -- one module in the whole corpus reached it. FIX: zap_inst the terminator before overwriting, at both sites. HONEST SEVERITY: the stale entry did NOT produce a wrong .osdi and no release build could be made to fail on it -- release never runs the MIR verifier (debug_assert!), and the passes that would trip over the record (replace_uses, use_set_value, whose bounds checks ARE active in release) happen never to touch that value again in any corpus model. A broken invariant with a latent release-crash hazard, not a demonstrated miscompilation -- and the last thing between the compiler and a clean assertions-enabled corpus run, the audit that produced E-286..293. A related latent case is deliberately left alone: update_inst_uses retires surplus use records with truncate, dropping them from the INSTRUCTION's list without detaching from the VALUE's -- same class, but unreachable today (every caller zaps first, and attach_use's own debug_assert would have fired otherwise), so it is documented rather than patched speculatively. Verify (examples/vafcodegen): staleuse.va compiles and simulates; the authoritative check is the assertions-enabled corpus replay -- all 255 models clean, 0 latent, where simctrl_demo.va aborted. Release-vs-release 255/255, 0 regressions; cargo test 69/0. Two source files.

E-295 *examples/vafautodiff_examples/*, examples/vafcodegen_examples/

regression guards for the two correctness blind spots (verification-only, no source change). A correctness campaign (~150 oracle checks over parameter storage, multi-terminal Jacobian/capacitance matrices, noise, node collapsing, \$mfactor, \$table_model, temperature) found ZERO defects; this folds in only the two checks that were genuinely NEW coverage. NOT added because already covered: flicker-noise 1/f and the correlated-noise summation rule (noise_examples + noisecorr_examples assert both exactly) and 2-D \$table_model (mdtable already checks the DC surface AND both partials). [1] FULL MULTI-TERMINAL MATRICES (vafautodiff, 16->18 checks): the suite biased only 2-terminal devices and [cross] read ONE off-diagonal, so the entries openvaf does not obtain by differentiating a contribution -- the KCL-derived source row, and the identically-zero row/column of an untouched terminal -- were untested; the new [matrix] checks measure all 16 entries of BOTH dI/dV and dQ/dV (separate reactive code path) on a device whose two contributions are polynomials in three distinct branch voltages at a bias where every branch voltage differs. MUTATION-TESTED: dropping the second product-rule term in mir_autodiff (d(uv)=u'v) makes both [matrix] checks FAIL (1.6e-1 / 2.2e-1) while [cross], [regression] and [multipoint] all still PASS -- the new guard has UNIQUE detection power, not just breadth. [2] PARAMETER SLOTS PER INSTANCE (vafcodegen, 17->19 checks): E-290 was a wrong struct-GEP offset, and in a 1-2 parameter model such an error lands on the right bytes by luck -- which every prior oracle test

was. `paramslots.va` interleaves 13 model+instance parameters of mixed types with distinct non-round values, mirrors each through its own opvar, and 3 instances across 2 model cards give 39 readbacks covering defaults, model card, instance line, instance-overrides-model and cross-instance isolation. MUTATION-TESTED: the slot index is used by BOTH writer and reader so permuting it is a self-consistent renaming and unobservable -- only a reader/writer MISMATCH shows (which is what E-290 was); making `nth_opvar_ptr` read a different slot than eval wrote yields `mp0 = 7.0` (its neighbour `ip0`'s value) and the guard fails. HONEST LIMIT found by that mutation test: E-290 fixed two sites, and only `load_eval_output` is reachable (covered by the existing ac_stim check); `nth_opvar_ptr`'s ParamKind::Temperature arm is NOT reachable -- `ov = \$temperature` lowers to a computed eval-output slot, and a reachability marker compiled into that arm was hit ZERO times across all 326 corpus models -- so it cannot be covered by a runtime test and the suite does not claim to. Regression 237/237. Two example suites, one new model.

E-307 *openvaf/sim_back/src/topology/lineralize.rs*

A **`ddt`** with no contributions crashed the compiler. Found by grammar-based fuzzing aimed at the MIDDLE/BACK end (the parser was already hardened; this generator emits well-typed Verilog-A that compiles, so it reaches MIR/optimizer/autodiff/codegen), run against the assertions-enabled build. lineralize.rs assumed an analog operator reaching the linearizer with an empty contribution list could only be noise: `assert!(noise, "ddt should have been deadcode eliminated")`. False -- a `ddt` whose result never reaches a contribution survives DCE. And it was a PLAIN assert!, not debug_assert!, so the SHIPPED release crashed ("OpenVAF encountered a problem and has crashed!") on valid input. 5 independent seeds of 3000 hit the identical assert; delta-debug + ablation pinned the trigger to ddt + a current probe on a declared (probe-only) branch + if/else + case, in a module contributing nothing -- removing any one stops it. Fix: return `Evaluation::Dead` unconditionally (the branch already taken for noise; its consumer replaces the result with zero and retargets pending uses), so a ddt that feeds no device equation contributes zero. Verify (examples/vafdeadop): reproducer compiles + .osdi loads + a CONTRIBUTING ddt still gives

E-308 *openvaf/mir_llvm/src/builder.rs*

Uninitialized read feeding a loop-carried phi crashed codegen. Second bug from the same grammar-based middle/back-end fuzz campaign as E-307 (seed 3230 of 8000). A variable read BEFORE a loop that is its only writer leaves the loop-carried phi with an incoming value no reachable block defines: MIR shows `v29 = phi [v18, block7], ...` where v18's defining phi was dropped by a pass on the dead path but the edge kept. Codegen's phi-completion loop then hit `BuilderVal::get()` on a still-`Undef` value -> `unreachable!("attempted to read undefined value")` at builder.rs:143 -- a plain unreachable, so the SHIPPED build crashed on valid Verilog-A. The MIR verifier misses it (its phi check only tests dominance when the def has a block; a detached def passes; and it is debug_assert! anyway). The trigger needs the module to contribute nothing keeping the value live (adding `I(p,n) <+ ra` fixes it). Fix, PROVABLY correct: build_func builds every reachable block before completing phis, so a phi input still Undef names a value NO reachable block defines (a dead path) -> lower it to `cx.const_undef` of the phi's type rather than panicking. Whole-class fix (any optimizer leaving a value that flows only into dead phi edges), not a per-pass chase. Verify (examples/vafuninitloop): reproducer compiles + a LIVE loop-carried accumulator still reads back exactly N*g (proves undef touches only dead inputs); fails on the pre-fix compiler; 328-model corpus identical pass/fail old vs new; 8000-seed re-fuzz shows 0 of this and the E-307 crash. THIRD, rarer pre-existing ICE surfaced and documented not fixed: packed_option.rs:60 PackedOption::unwrap() on None, ~1/8000, old compiler crashes too.

E-309 *openvaf/mir_opt/src/global_value_numbering.rs*

GVN crashed on a user instruction in an unreachable block. Third and final crash from the same grammar-based middle/back-end fuzz campaign as E-307/E-308 (seed 6716, ~1/8000). When

an instruction's congruence class changes, GVN re-queues its USERS via `inst_to_dfs[user].unwrap_unchecked()`. DFSMapping::populate numbers only instructions reachable through cfg_postorder, so a user in an UNREACHABLE block has no DFS id. `unwrap_unchecked` = `if cfg!(debug_assertions) { self.unwrap() } else { self.0 }`, so it panicked at packed_option.rs:60 under debug-assertions and in release returned the reserved sentinel that `touched_insts.insert` used as an OUT-OF-RANGE BitSet index -- the SHIPPED compiler crashed either way ("OpenVAF encountered a problem and has crashed!"). Fix: skip users with no DFS id -- an unnumbered user is in an unreachable block, not in the GVN work list (the solver only iterates dfs_to_inst), so re-queuing it is a no-op -- exactly as `get_rank` in the same file already tolerates the identical None (`if let Some(dfs_id) = inst_to_dfs[inst].expand()`). The two other unwrap_unchecked sites operate on the current instruction (always numbered) and are unchanged. Verify (examples/vafgvnunreach): reproducer compiles + a CSE-heavy model GVN actively optimises still computes exact I = 4*V*g + (V*g)^2 (proves the pass is undisturbed); fails on the pre-fix compiler; 330-model corpus identical pass/fail old vs new. A 12000-seed re-fuzz against the fully-fixed compiler shows 0 of this and the E-307/E-308 crashes -- the three distinct crashes this campaign found are all closed. That deeper run also tripped ONE `debug_assert!(cx.func.validate())` at sim_back/src/lib.rs:175 (seed 11633): NOT a shipped crash (release compiles it fine), but the same assertions-only malformed-MIR class as E-286..E-294, documented for a separate follow-up.

E-310 *openvaf/mir_opt/src/simplify_cfg.rs*

Constant-branch fold left an SSA-invalid phi (the sim_back/lib.rs validate assert). The MIR-validity defect the E-307/308/309 fuzz campaign surfaced, now RESOLVED. const_fold_terminator folds ‘br TRUE

E-313 *openvaf/hir_ty/src/inference.rs*

Two builtin argument type-coercion gaps, both emitted silently by the release compiler. Found by a fresh round of the same grammar-based middle/back-end fuzz campaign as E-307..310, run against the assertions build. (a) FILE/STRING FORMAT TASKS were never type-checked. `infere_display` parses the format string and inserts the int->real cast a `%g/%e/%f/%r` conversion needs, but was reached only by the CONSOLE tasks (`\$display/\$strobe/\$write/\$monitor/\$debug + \$fatal/\$warning/\$error/\$info`); the FILE tasks (`\$fdisplay/\$fwrite/\$fstrobe/\$fmonitor/\$f`) and STRING tasks (`\$swrite/\$sformat`) were missing from the dispatch match, so their format args were unchecked. A `%g` fed an integer kept its integer value while `print_callback` (osdi/compilation_unit.rs) types the callback parameter as `double` from the conversion -- so lowering passed a raw i32 to a double parameter: INVALID LLVM IR (`Call parameter type does not match function signature! i32 3 / double %35 = call ... @cb.2(..., i32 3)`). The verifier that catches this is a `debug_assert!` (`llmod.verify_and_print()`), compiled out of release, so release emitted a malformed .osdi whose callback reads the integer's bit pattern as a double -- garbage. Observable: a `\$sformat(s,"%g",5)` -> `\$sscanf(s,"%g",g)` round-trip used as a conductance reads back the denormal 2.47e-323 (bits of the integer 5) instead of 5, so a 5*V device collapses to ~0. Console tasks were unaffected (they carry the cast). Fix: add the file/string format builtins to the infere_display dispatch arm -- infere_display scans for string-LITERAL format strings, so the leading fd (integer) or destination (string variable) argument is naturally skipped and the real format string found; the console path's exact logic now applies to every format task. (b) `ddx` WITH AN INTEGER ARGUMENT crashed the compiler. infere_ddx requires its first argument (the differentiated value) to be real via `self.expect::<false>(expr, None, ty, [Val(Real)])` -- but `expect` records the needed cast on its FIRST parameter, and it was handed `expr` (the whole ddx call) while `ty` is the type of `val` (the first argument). For an integer `val`, expect recorded an int->real cast ON THE ddx CALL expression, which already has type Real; `needs_cast` then computed src=Real (the call's type) and dst=Real (the recorded cast) and tripped `debug_assert_ne!(src, dst, \"cast types must be different\")` at hir/src/body.rs -- with the assert compiled out, the release build aborted downstream with no .osdi (`ddx(n, V(b))`, integer n). Fix: record the requirement on `val`, the argument, not on `expr`; an integer argument is then coerced

*to real (ddx of a probe-independent value is 0) and a real argument is unchanged. Both fixes only touch previously-crashing/invalid paths: the whole 419-model corpus produces BYTE-IDENTICAL MIR before and after (deterministic --dump-mir oracle, 0/419 changed), the corpus replays clean through the assertions build, and a 15000-module re-fuzz on the fixed compiler is clean. Verify (examples/vafargcoerce): 4 checks under both solvers, all failing on the pre-fix binary -- ddx(integer) compiles + simulates to I=1e-3*V, and $sformat("%g",integer) compiles + the round-tripped value is exactly 5. DEFERRED (same campaign, needs a design decision): a provably-infinite analog loop (`while (1) ...`) crashes the compiler via a degenerate no-reachable-exit CFG -- a DCE guard stops the first unwrap but the crash resurfaces in the CFG-validity machinery; left for a dedicated change (reject non-terminating analog loops with a diagnostic, or make the passes tolerate an unreachable exit).

E-314 *openvaf/hir/src/elaborate.rs, openvaf/mir_opt/src/const_eval.rs, openvaf/hir_ty/src/inference.rs*

Constant-evaluation / literal-materialization robustness -- one shipped DoS, two overflow aborts. From the E-307..313 grammar-fuzz family. (a) INTEGER CONST-FOLD OVERFLOW (three sites, one class): two hand-rolled integer const evaluators used UNCHECKED i32 arithmetic. elaborate. rs's Enhancement-91 bus-width folder had checked_mul but its parse_add +/- (`acc += / acc −=`) and parse_unary negate (`−parse_unary`) were unchecked, so `localparam integer k = 2147483647 + 1;` (with any `[...]` declaration present -- the folder is gated on the module text containing '[') overflowed; and mir_opt/const_eval.rs, where Enhancement-286 made Iadd/Isub/ Imul wrapping and noted 'eval_unary already used this convention' but MISSED Opcode::Ineg, so negating i32::MIN (from `−(1<<31)`) overflowed. All three aborted the overflow-checked build; the shipped release wrapped silently. Fix: elaborate.rs uses checked_add/checked_sub/checked_neg -- declining the fold on overflow exactly as its ∗ already did, and fold_parameter_widths handles None by leaving the declaration unchanged; const_eval.rs uses val.wrapping_neg(), matching its own wrapping binary ops. (b) UNBOUNDED REPLICATION -> SHIPPED COMPILE-TIME DoS: `{N{..}}` materializes N copies at COMPILE time (infere_concat/lower_string_concat build an N*

E-317 *openvaf/mir_llvm/src/builder.rs*

`idt` initial-condition in a dead branch crashed codegen. From the E-307..314 analog-operator fuzz family. An idt(,*IC) or idtmod placed inside a statically-false branch -- `if (ceil(0) > 1) w = idt(V(a),0);` -- crashed the shipped compiler (exit 101). ceil(0)>1 is always false but ceil() is NOT const-folded, so the dead branch survives into MIR; because w's idt initial-condition state is never used, codegen prunes the branch CONDITION's computation as dead, yet the Branch instruction survives into the derived osdi::setup::setup_instance function. When build_func reaches that branch and reads its condition, the condition's BuilderVal is still Undef, and BuilderVal::get hit unreachable!("attempted to read undefined value") (mir_llvm/builder.rs:143). Only idt/idtmod (integrator accumulator STATE with an IC) trigger it -- ddt/absdelay/transition/slew/laplace*/bare idt(x) are clean. Same Undef-value class as Enhancement-308 (which handled the PHI-INPUT case); this is the BRANCH-CONDITION case. Fix: when a Branch's condition is Undef, lower it as constant false -- the branch only guards dead code (the unused idt-IC state init), so the guarded path never executes on either edge, making this observationally equivalent and avoiding an undefined `br`. Verified corpus-bit-identical (only the reproducer, which was crashing, changes; the other 419 models are byte-identical MIR). Verify (examples/vafidtcfg): reproducer compiles (crashed before) + simulates to a finite op (i(v1)=-5e-4). DEFERRED from the same hunt: an assertions-only PHI-type mismatch (an uninitialized integer variable's default materialised as f64 F_ZERO -> i32-result phi with an f64 operand; release folds both to 0, correct) and a ngspice .tran convergence livelock on a specific fuzzer OSDI device (tiny 1e-15 cap + clamped exp diode + tanh + noise) -- both left for dedicated changes.

E-324 *openvaf/hir_lower/src/expr.rs*

`\$fatal` stranded code in an unreachable block -- TWO shipped-compiler crashes, one root

cause. Found by a diverse-strategy fuzz campaign against the assertions build (~75000 generated well-typed models, 7 generation strategies); both crashes reproduce on the SHIPPED release, not just the instrumented build. `analog begin \$fatal(0); I(a,c) <+ V(a,c)/1k; end` panicked at mir_opt/src/dead_code_aggressive.rs:112 (`Option::unwrap()` on `None` from `self.func.layout.inst_block(inst).unwrap()` in mark_inst_live), and the mirror image `analog begin I(a,c) <+ V(a,c)/1k; \$fatal(0); end` panicked at mir_llvm/src/builder.rs:143 (`unreachable!("attempted to read undefined value")` in BuilderVal::get, reached from store_residual). ROOT: `\$fatal` lowered to print -> set the abort flag -> `exit()` -> create a fresh block with NO incoming edges -> keep lowering into it, on the assumption (stated in its own comment) that an edge-less block is simply removed from MIR. That is unsound for a compiled device: every ret-flag (RetFlag::Abort/Finish/Stop) is only a flag the simulator inspects AFTER the eval function returns -- all three lower to `set_ret_flag_*` callbacks in osdi/src/compilation_unit.rs -- so none can jump out of the middle of an evaluation, and the OSDI eval function has a MANDATORY epilogue (store residual/jacobian) the ABI requires to run. Terminating the MIR function early therefore stranded work in the predecessor-less block: a statement AFTER `\$fatal` was lowered there yet stayed referenced by the contribution bookkeeping (values held OUTSIDE the DFG are not reached by CFG cleanup -- the E-294 class), so aggressive DCE marked live an instruction belonging to no block; and with a statement BEFORE it the EPILOGUE ITSELF was emitted there, where the residual value does not dominate, so codegen read an Undef. `\$finish` and `\$stop` were already lowered correctly as set-flag-and-continue (no exit(), no unreachable block) -- `\$fatal` was the lone outlier. FIX: `\$fatal` sets its flag and falls through exactly like `\$finish`/`\$stop`; the CFG stays connected so neither failure mode can arise (one hunk, one file). Measured trigger surface: crashes required an UNCONDITIONAL `\$fatal` plus a contribution in the module -- a conditional `if(..) \$fatal(0);` never crashed (the join block is still reachable via the else path), nor did `\$fatal` with no contribution, nor `\$finish`/`\$stop` in any position. BEHAVIOUR UNCHANGED: `\$fatal`'s run-time meaning lives entirely in the set_ret_flag_fatal callback, untouched -- simctrl_examples (E-55) passes in full including "message printed", "abort error printed", "transient aborted early" and "parameter-only $fatal: setup rejected"; vafcodegen_examples (E-294) 19/19 including the parameter-guarded `\$fatal` arm which still SIMULATES correctly (I=V/1k); and a realistic guard (`if (r<=0) \$fatal(..)` followed by a `V/r` divide) still aborts cleanly at setup with its message and no NaN leaking. The one semantic difference -- statements after `\$fatal` now execute instead of being dead -- is exactly what `\$finish`/`\$stop` have always done and is not observable, since the simulator aborts as soon as eval returns. OUTPUT-PRESERVING: `--dump-mir` (deterministic, unlike .osdi bytes) over the full 462-model corpus is BYTE-IDENTICAL for 460; the only two that differ are precisely the models using `\$fatal` in a reachable analog context (simctrl_demo.va, staleuse.va), both of whose suites pass in full, and the two hisim2 industry models that also use `\$fatal` are unchanged. Verify (examples/vaffatalcfg): the two crash shapes compile (both fail on the pre-fix binary with a compiler panic and no .osdi) plus run-time behaviour guards; 6 checks.

E-325 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs*

Bound the MATERIALIZED SIZE of **`{...}`**/**`{n{...}}`**, not just the replication count -- a shipped compile-time HANG and a shipped silent WRAP. Enhancement-314 capped the replication COUNT at 2^20 after a `{'d999999999{"x"}}` DoS, but the count is only ONE FACTOR of the final size, so two abusive shapes still reached the shipped compiler (found by the 7-strategy ~75000-model fuzz campaign). (a) STRING replication becomes LLVM FUNCTION ARITY: `lower_string_concat` builds an `elems.len()*rep_cnt` operand list plus a format string with that many `%s`, and `print_callback` turns it into a generated callback with ONE PARAMETER PER OPERAND. LLVM degrades super-linearly in arity -- measured on this compiler: 2000 operands 0.4s, 8000 2.9s, 16000 8.6s, 32000 never finished -- so `parameter string s = {200000{"x"}};` HUNG the compiler on one line of source (both binaries). NOTE the original hypothesis (quadratic string interning in the const-folder) was WRONG: the const-fold is linear and negligible; the entire cost is in LLVM. (b) The size arithmetic OVERFLOWED u32: `len: total * rep_`

`cnt` was an unchecked multiply (and the running `total += len` equally unchecked), so `real c[0:1]; c = {1048576{{1048576{1.0}}}};` -- where BOTH counts are individually legal (exactly MAX_REP) -- computed 2^20*2^20 = 2^40, which panicked under overflow-checks and in the SHIPPED release silently WRAPPED TO 0, emitting the nonsense diagnostic `expected real[0:2] value but found real[0:0] value` instead of a real error. FIX: compute the size in u64 with saturating add/mul and bound it BEFORE narrowing to the u32 array length, with a dedicated `ConcatTooLarge` diagnostic that reports the TRUE expanded size (`expands to 1099511627776 elements` = 2^40, where the u32 path had wrapped to zero). Reusing the existing InvalidReplicationCount diagnostic would have been MISLEADING -- in case (b) the count IS a valid positive integer literal; it is the product that is too large. TWO limits, because the two paths have genuinely different measured cost profiles: MAX_CONCAT_ELEMS = 2^20 for numeric arrays (that path is linear and cheap -- 65536 elements compile in 0.41s -- so this only rules out the absurd and guards the u32 length) and MAX_CONCAT_STR_OPERANDS = 4096 for strings (arity is the real cost; 4096 keeps the worst case near a second, orders of magnitude above any legitimate literal). LRM POSITION: `{n{...}}` over strings/1-D arrays is this project's Enhancement-34 EXTENSION -- Verilog-A (LRM Annex C) has no vector concatenation at all and Verilog-AMS defines `{}`/`{n{}}` over bit vectors -- so no LRM-mandated semantics is violated by a documented expansion limit (the precedent E-314 set); and for (b) any expansion larger than the destination is a TYPE ERROR today, the only defect being that the compiler computed the size wrongly before it could say so. No legal program changes meaning. OUTPUT-PRESERVING: u64 saturating add/mul agree exactly with the old u32 wrapping arithmetic for every value below 2^32 and both caps are checked before the narrowing, so every model whose concatenation fits takes a bit-identical path -- confirmed with the deterministic `--dump-mir` oracle over the 465-model corpus: 464 byte-identical, and the single reported diff (lrm_p150_1.va, which contains NO concatenation at all) was PROVEN to be run-to-run nondeterminism of the multi-module dump order, not an effect of the change (the same binary on the same input produced both hashes across 5 runs). Verify (examples/vafconcatsize): the three abusive shapes are rejected cleanly (they hang or misreport on the pre-fix binary), the diagnostic reports the true 2^40 size, and legitimate concatenation still compiles AND simulates (a replicated array element drives i(v1) = -2 mA exactly); 6 checks.

E-326 *openvaf/sim_back/src/init.rs*

A cross-namespace **`mir::Value`** comparison mis-typed init cache slots -- the shipped compiler SIGSEGV'd inside LLVM. Highest-severity finding of the 7-strategy ~75000-model fuzz campaign: on a legal Verilog-A model the RELEASE binary died with EXC_BAD_ACCESS in `llvm::detail::DoubleAPFloat::multiply` (signal 11, no diagnostic); the assertions build instead had its LLVM verifier reject the module with `fmul reassoc .. i8 %8, double %6` and `trunc double %45 to i8`, and at -O0 release gave `LLVM ERROR: Do not know how to promote this operator!`. A boolean-typed (i8) value was reaching floating-point arithmetic. ROOT: NOT a missing cast, NOT autodiff, NOT casex. A `mir::Value` is a bare u32 index and the MAIN function and the INIT function each number their values from zero. `build_init_itern` inserts `self.val_map[&val]` -- an INIT-namespace value -- into `collapse_implicit`, which is correct for its other consumer `optimize()` (that runs DCE over `self.init.func`); but `build_init_cache` iterates `self.init_cache`, whose keys are MAIN-namespace values, and tested them against that same set (`collapse_implicit.contains(&val)`) both in its dead-value filter and in its type selection. Comparing indices across two namespaces cannot fail loudly -- it silently SUCCEEDS whenever the two counters coincide. Confirmed under lldb on the minimized reproducer: the producer inserted init `Value(25)` (the `idt` collapse flag, a genuine Bool) while the consumer tested main `Value(25)`, which is `abs(p)` -- the NOISE POWER, a Real. Same number, unrelated values. CONSEQUENCE: a colliding f64 slot was recorded as `Type::Bool`, which lowers to `ty_c_bool()` = i8. The store side (`store_cache_slot`) then int-cast a slot it believed boolean, emitting `trunc double .. to i8`; and the noise loader reads `EvalOutput::Cache` RAW with the slot type -- unlike `load_cache_slot` it performs no bool normalization -- so the i8 became `base_pwr` and

landed in `fmul i8 %x, double %y`. LLVM constant-folded that malformed fmul and read the i8 as an APFloat: the EXC_BAD_ACCESS. FIX: map through `val_map` before the lookup so both sides speak the same namespace -- 'let is_collapse_flag = self.val_map.get(&val).map_or(false,

E-327 *openvaf/hir_lower/src/{expr.rs,ctx.rs,lib.rs}*

`ddx` unknowns that do not lower to a bare MIR **`Param`** crashed the shipped compiler -- the bug was sitting behind a **`TODO`**. From the 7-strategy fuzz campaign. `ddx` lowering assumed its UNKNOWN argument always lowers to a bare `Param` and unwrapped one unconditionally: the DDX_POT arm did `let node = self.ctx.unwrap_node(unknown);` (itself `value_def(val).unwrap_param()`) directly under a `// TODO how to handle gnd nodes?` comment, and the other arm did `CallBackKind::Derivative(self.ctx.dfg().value_def(unknown).unwrap_param())`. That assumption is false: `LoweringCtx::nodes` yields THREE shapes -- a `Param` for a forward-oriented probe (`V(a,b)`, `V(a)`); an **`fneg(param)`** INSTRUCTION for a REVERSE-oriented probe (`V(b,a)`, or one whose high side is ground, via either the `(None, Some(lo))` arm or the inverted-param arm); and **`F_ZERO`**, a constant, when the probe is ground only. The latter two hit `unwrap_param()` and panicked `Value is not a parameter` (mir/src/dfg/values.rs) on BOTH binaries -- a shipped crash on legal input. DECISION -- COMPILE, do not reject: both extra shapes have an unambiguous derivative. `V(b,a)` denotes the SAME branch with the opposite reference direction, i.e. `V(b,a) == -V(a,b)`, so `df/dV(b,a) == -(df/dV(a,b))`; and ground is not an unknown of the DAE system, so `df/dV(gnd) == 0` -- exactly what the backend already does for a derivative callback that never became an unknown. Erroring would also be incoherent: openvaf already ACCEPTS `V(a,b)` as a ddx unknown (an extension beyond the LRM's single-net/branch-flow rule, announced by its own L011 lint), so having accepted `V(a,b)` it cannot reject `V(b,a)`. FIX: peel an `fneg` off the unknown (recording that the result must be negated), then ASK rather than assert whether what remains is a parameter -- `match self.ctx.dfg().value_def(probe).as_param() { None => F_ZERO, Some(param) => ... }` -- negating the callback's result when an fneg was peeled; plus a non-panicking `ParamKind::pot_node()` replacing `unwrap_pot_node()` on the DDX_POT path and a `LoweringCtx::param_kind()` accessor so lowering can inspect a user-supplied unknown instead of asserting its shape. VERIFIED NUMERICALLY, not merely that it compiles: differentiating `V^2` (exact derivative 2V) at V=3 through a 1 mS scale gives i = -6.00000e-03 for the forward unknown `V(a,b)`, i = +6.000000e-03 for the reverse unknown `V(b,a)` -- the EXACT negative to machine precision -- and -3.00000e-09 for the ground unknown, which is purely the model's explicit 1e-9*V leak term, i.e. the ddx contributed exactly zero. OUTPUT-PRESERVING: the new code diverges from the old only when `value_def(unknown)` is NOT a `Param`, which is precisely the case that previously reached `unwrap_param()`/`unwrap_node()` and panicked, producing no output at all; when the unknown IS a Param (every model that compiles today) the fneg peel does not fire and the callback construction is unchanged. Confirmed with the deterministic `--dump-mir` oracle over the corpus: 0 changed. Verify (examples/vafddxunknown): the two crashing shapes compile and their derivatives are checked against the closed form; 4 checks.

E-328 *openvaf/hir/src/body.rs, openvaf/hir_lower/src/expr.rs*

`BodyRef::get_expr` was a PARTIAL function used as if it were total -- a dynamic array index in a contribution crashed the shipped compiler. `get_expr` funnelled every `BitSelect` into `resolve_path`, which can only resolve expressions that have a `Ref` (Ty::Var, Ty::Param, a function var/return, a nature attribute) and `panic!`s otherwise. A dynamically-indexed array read has NO backing variable: inference types it `Ty::Val(..)` and records the element variables, per-dimension bounds and index expressions OUT-OF-BAND in `dynamic_index_refs`. So any caller that merely probed an expression's SHAPE -- a literal-zero test, a literal-condition fold, an aggregate check -- crashed on legal input with `invalid HIR: path BitSelect { .. } was not resolved Val(Real)`. The asymmetry is the tell: `x = g[k]; I <+ V*x;` compiled while `I <+ V*g[k];` panicked -- `lower_expr` short-circuits on `dynamic_index()` BEFORE consulting `get_expr`, and the contribution path has no such short-circuit. (Note an earlier analysis reported this

shape as already working; re-verification on the shipped binary showed it still crashed in both the uninitialised-index and assigned-index spellings -- worth re-testing rather than trusting.) FIX: restore the invariant that `get_expr` is TOTAL -- add an `Expr::DynIndexRead` variant and answer the shape question directly, gated on the same `dynamic_index_refs` map the value path already consults. The VALUE is still lowered by `lower_expr`'s existing `dynamic_index()` short-circuit, so nothing about code generation changes. Making the enum non-exhaustive pointed the compiler at EXACTLY ONE match needing an update (`lower_expr`'s, which already short-circuits and so gets an `unreachable!` arm), confirming how tightly scoped the change is. VERIFIED NUMERICALLY: a dynamic index must select the RIGHT element, not merely stop crashing -- four instances selecting g = {1,2,3,4} mS at V=1 give exactly -1.00000e-03/-2.00000e-03/-3.00000e-03/-4.00000e-03, agreeing bit-for-bit with the `x = g[k]` spelling that always worked. OUTPUT-PRESERVING: the new branch is gated on `dynamic_index_refs.contains_key(&expr)`; for any expression not in that map -- every expression in every model that compiles today -- `get_expr` executes exactly the previous code path, and for an expression that IS in it the previous behaviour was a panic, so there is nothing to preserve. Confirmed with the deterministic `--dump-mir` oracle. Verify (examples/vafdynidx): 3 checks.

E-329 *openvaf/sim_back/src/topology/small_signal_network.rs*

A GRAVESTONE phi operand crashed the small-signal network builder. `ddt` of a NEGATED flow probe plus a statically-false branch containing a loop -- `r0 = ddt(-I(a,b)); if ((1*s0)-s0) begin lc3=0; while (lc3<4) lc3=lc3+1; end I(b,c) <+ r0*r1;` -- panicked the SHIPPED compiler with `internal error: entered unreachable code`. ROOT: the value arriving is GRAVESTONE, `Value(0)`, whose `ValueDef` is `Invalid`. It is the compiler's OWN placeholder, declared in mir/src/dfg/values.rs as "place holder for unused values that must remain (in phis)": the SSA re-builder puts it in a phi for an edge with NO reaching definition -- an edge from a block unreachable from the entry that `simplify_cfg` cannot delete. The small-signal network builder asserted such a value could never reach it, with `ValueDef::Invalid => unreachable!()` in BOTH `analyze_value` and `analyze_dependency`. FIX: a GRAVESTONE operand sits on a dead edge and therefore cannot be used at run time, so it contributes nothing -- `analyze_value` returns `FlatSet::Zero` (the same answer its neighbouring F_ZERO arm gives) and `analyze_dependency` returns `Dependency::Independent` (the same answer its 'Param(_)

E-330 *openvaf/hir_ty/src/validation/body.rs*

`ddx` in a runtime loop HUNG the shipped compiler forever -- a fixpoint that grows its own lattice. The last of the seven shipped crashes from the diverse-strategy fuzz campaign, and the only infinite loop rather than a panic: `for (i=0;i<1;i=i+1) x = ddx(V(a)*x, V(a));` never returned on either binary. ROOT: `live_derivative_fixpoint` (mir_autodiff/src/live_derivatives.rs) asks, for every derivative already live at a ddx call, for one of ORDER+1 via `raise_order_with`. That is fine on a DAG -- the order chain is bounded by the number of ddx sites on the longest path. A loop back edge closes the circuit: the differentiated expression `V(a)*x` depends on `x`, which is the loop-carried phi fed by the ddx result itself, so round n creates order n+1 and interns it, `populate_reachable` marks it reachable at that instruction, the changed live set requeues the argument's defining instructions, and the back edge carries the new order around the phi back into the ddx's own input -- round n+1 begins. A monotone fixpoint terminates because its lattice is FIXED; here the fixpoint GROWS THE VERY LATTICE IT ITERATES OVER, so it has no fixed point. Measured not inferred: `sample` puts 99.8% of 3944 samples in that one chain (raise_order_with -> TiSet::ensure -> IndexMap::insert_full), RSS climbs monotonically and never plateaus, and the process was still running after 15 minutes -- true non-termination, not slowness. DECISION -- clean error, not a cap: ddx is SYMBOLIC and MEMORYLESS (openvaf computes it as a derivative callback on the MIR; its result is determined by the SHAPE of its argument). Inside a runtime loop `x = ddx(V(a)*x, V(a))` asks for a different symbolic form every trip -- iteration k needs the k-th derivative and the trip count is not a compile-time constant -- so THERE IS NO FINITE MIR THAT IMPLEMENTS IT, which is exactly why the fixpoint diverges rather

than converging to something wrong. Ill-formed, not legal-but-unbounded. This is also what the language already says and what the compiler already does for every OTHER analog operator: ddx is classified as an analog operator by openvaf itself (hir_def/src/builtin.rs is_analog_operator()) and LRM 4.5.1 forbids analog operators in non-genvar loops -- `for(...) x = ddt(V(a));` and `for(...) x = idt(V(a)*x);` already produce `analog operator ... is not allowed in loops`. The divergence came from one explicit escape hatch in hir_ty/src/validation/body.rs, `_ if call.is_analog_operator() && call != BuiltIn::ddx`, which is CORRECT for conditionals (the industry CMC corpus has 192 ddx call sites inside `if` bodies across 19 models -- BSIM-BULK, BSIM-SOI, HICUM, ASM-HEMT, L-UTSOI) but too broad, since it also covers loops. FIX: track runtime-loop nesting in a dedicated `loop_depth` counter and reject ddx there, reusing the existing IllegalCtxAccess diagnostic so it is an ordinary compile error (exit 65, no new machinery). A separate counter is required rather than reusing `ctx`: `validate_condition_in` REPLACES the context instead of stacking it, so an `if` nested in a `for` resets it to Conditional, and it only becomes Loop when the controlling expression is non-constant, so `repeat(3)` would slip through. Trigger surface verified: `for`, `while` and `repeat` all hung, as did a ddx under an intervening `if` and one routed through a second variable; the multiplication is NOT essential (`ddx(V(a)+x, V(a))` hangs too) -- the precise condition is that the argument depends on the differentiation unknown AND on the ddx call's own result through a back edge. SCOPE, STATED PLAINLY: ddx outside a loop is untouched, including inside `if/else`, and is still numerically exact there (d/dV(V^2)=2V gives -6 mA at V=3). A corpus scan of 514 models finds 0 of 755 ddx call sites inside a loop body, and the --dump-mir oracle reports no corpus model changed. This is nonetheless the ONE fix in this series that slightly NARROWS the accepted language: a loop containing a ddx with no self-reference (`for(...) g = g + ddx(V(a)*V(a), V(a));`) compiles today and now errors. It is absent from the corpus, the rejection is LRM-conformant, and it is exactly the treatment ddt/idt/transition/laplace_* already receive -- but it is not strictly output-preserving and is not claimed to be; keeping it would require front-end dataflow that can distinguish self-referential from not, which the validator does not have. SEPARATELY DISCOVERED, NOT FIXED HERE: while bounding the derivative order a SECOND shipped crash surfaced -- 65 nested ddx calls panic in lib/bitset/src/lib.rs (`index out of bounds: the len is 1 but the index is 1`) via HybridBitSet::contains, reached from populate_reachable; a row that has gone dense keeps the word count it had at that moment, so once the derivative universe grows past 64 (one word) a query on the new index panics. Independent of loops and of this fix, left for a dedicated change. Verify (examples/vafddxloop): the hanging shape is now a prompt error citing the LRM, and ddx outside a loop still compiles AND stays exact; 4 checks.

E-331 *lib/bitset/src/lib.rs*

`BitSet::contains` panicked outside its domain -- a representation leaking out as a compiler crash. Found while bounding the derivative order for E-330: deeply nested `ddx` crashed the SHIPPED compiler with `index out of bounds: the len is 1 but the index is 1` at lib/bitset/src/lib.rs:114, via `BitSet::contains` <- `HybridBitSet::contains` <- `mir_autodiff::populate_reachable`. ROOT: `contains` indexed its backing storage with no bounds guard -- `(self.words[word_index] & mask) != 0`. A `HybridBitSet` stores a set either SPARSELY (a Vec of elements) or DENSELY (a bit array), switching as a size optimisation, and rows grow their domain only when something is inserted into them -- so a row the traversal never inserts into keeps whatever width it had when it went dense. Once the live-derivative universe grew past 64, i.e. ONE WORD, a query for a higher element indexed off the end. The decisive detail is the ASYMMETRY between the two representations: `SparseBitSet::contains` searches a Vec and returns `false` for any absent element, while the dense `BitSet::contains` panics -- so the SAME logical query on the SAME logical set returned false while the set happened to be sparse and CRASHED once it happened to be dense. The representation, the very detail HybridBitSet exists to hide, leaked out as a crash. FIX: make the dense query total, matching what the sparse one already does -- 'self.words.get(word_index).map_or(false,

E-332 *openvaf/sim_back/src/topology/builder.rs, openvaf/hir_ty/src/validation/body.rs*

Summing THREE OR MORE **`ddt()`** terms into one contribution silently DROPPED CHARGE -- a wrong number, not a crash. `I(a,b) <+ ddt(V)+ddt(V)+ddt(V)` compiled cleanly and behaved as a 1 F capacitor instead of 3 F; N=1 and N=2 were correct, which is why it survived. Reachable from entirely ordinary code: a `generate` loop is unrolled at compile time, so `generate k (0,2) s = s + ddt(V);` is the same expression and was equally wrong. ROOT CAUSE was NOT in `ddt` handling but in TRAVERSAL ORDER. `create_dimension` replays the instructions consuming an analog operator's result to build its linear coefficient, and its Fadd/Fsub arms '(None, Some(x))

E-333 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs*

Integer division by a LITERAL zero compiled with exit 0 and no diagnostic, then killed ngspice with SIGTRAP and no output at all. `if ((5 / 0) > 0 && ione > 0) ...` -- LLVM treats `sdiv x, 0` as IMMEDIATE UNDEFINED BEHAVIOUR: the result folds to poison, poison reaching a branch becomes `unreachable`, and `unreachable` lowers to a `brk`, so the whole function became a trap and the first call into the compiled .osdi killed the host process. The short-circuit && is what made it reachable -- with a RUNTIME right operand the division lands in a conditionally-executed block where folding never collapses it, so it survives into code generation; the same expression without the && folded away, which is why it looked harmless. Every spelling was in fact undefined and only some trapped: before the fix `V(o) <+ (5/0) > 0 ? 1.0 : 2.0;` printed 0, which is not that expression's value but poison propagating. WHY E-286 LEFT IT: E-286 fixed a COMPILER crash here (folding 5/0 evaluated the division inside openvaf and killed it with an internal error) by declining to fold, reasoning that "a runtime zero divisor has always been accepted, so a literal one must be too". BOTH HALVES ARE WRONG -- (1) the literal case is not the runtime case, since only the literal one leaves a constant-zero divisor in the IR for the optimiser to exploit (verified: a parameter, a localparam and a derived constant `3-3` are all lowered as RUNTIME values and none of them traps, so the literal is the entire UB surface); (2) runtime acceptance is TARGET-SPECIFIC -- AArch64 returns a value where x86 raises SIGFPE, and this project ships x86 builds for macOS, Linux and Windows plus riscv64 among its targets, so there is no portable value to fold to. E-286 thus converted a compiler crash into a simulator crash. FIX: reject it, since no value can be defended -- a clean `error: integer division by zero` naming the zero and stating that a parameter/localparam zero is still accepted. The check is LITERAL-ONLY by design: that is exactly the set of programs reaching LLVM with a constant-zero divisor, and widening it to any folded zero would reject working models for no safety benefit. VERIFIED: every previously trapping or undefined spelling (inside &&, inside '

E-334 *openvaf/hir_ty/src/inference.rs, openvaf/hir_ty/src/diagnostics.rs*

E-333 was INCOMPLETE: the same SIGTRAP survived for **`i32::MIN / -1`** and for an out-of-range shift distance. `if ((-2147483647 - 1) / (-1) > 0 && ione > 0) ...` and `if ((1 << 40) > 0 && ione > 0) ...` both compiled with exit 0 and no diagnostic, then killed ngspice with signal 5 and no output -- identical mechanism to the zero divisor E-333 fixed, only the operation differs. HOW IT WAS MISSED: Enhancement-286's own comment named ALL THREE cases ("a zero divisor -- and i32::MIN / -1, which overflows", "a shift distance outside 0..32 is poison in LLVM") and declined to fold each, which is exactly what leaves the poison in the IR for the optimiser to turn into `unreachable` -> `brk`. E-333 read that comment, fixed the divisor, and did not check whether the other two it named had the same consequence. When a comment enumerates several cases sharing a mechanism, fixing one is not evidence about the others. FIX: the same front-end check extended to both operations, with a diagnostic each ("integer division overflows", "shift distance out of range" naming the distance). E-333's check tested for a LITERAL, which is insufficient here: i32::MIN CANNOT be written as an integer literal -- `-2147483648` exceeds i32::MAX and promotes to REAL, making E-286's `u = -2147483648 / -1` a real division rather than the integer overflow it was believed to test. The overflow is only reachable as `(-2147483647 - 1)`, so the check now folds constant integer expressions (literals plus + - * over them, wrapping arithmetic, bounded

recursion) -- exactly the set the code generator sees as constant. It stays CONSTANT-OPERAND-ONLY: a parameter or localparam is a runtime value, never a constant operand in the IR, and rejecting those would break working models. VERIFIED: every previously trapping shape is now a clean exit 65 -- i32::MIN/-1, the same with %, 1«32, 1«40, (-1)»32, 1«(-1), and 1«(4*8) (which needs the folding, not just a literal); and everything legal still compiles AND simulates -- 1«31, 1«0, a runtime shift distance, parameter/localparam zero divisors, (-2147483647-1)/2, ordinary division and remainder, I = V/1k exactly. CORPUS: 480 models, ONE changes accept/reject -- E-286's constfold.va, whose `1 << 40` moved to the new negative test and was replaced by the largest LEGAL distance (1«31); its `-2147483648 / -1` line stays with a comment recording that it is a REAL division and never tested the integer overflow it was written for. STILL OPEN, not addressed: at RUNTIME an out-of-range shift distance is silently masked to 5 bits (`1 << n` with n=32 yields 1 where Verilog requires 0) -- a wrong answer rather than a trap, needing a codegen change rather than a front-end check. Verify (examples/vafintub): 6 checks.

E-335 *openvaf/mir_llvm/src/builder.rs, openvaf/mir_opt/src/simplify.rs*

Three IEEE 754 / IEEE 1364 semantics the compiler was not honouring -- silent wrong answers on ordinary code, no crash. (1) `!=` ON REALS WAS AN ORDERED COMPARE: `Fne` lowered to LLVMRealONE, which is FALSE whenever an operand is NaN, so `x != x` -- the canonical isnan idiom -- reported "not NaN", and `a != b` was NOT the complement of `a == b` on the same operands. `Feq` correctly stays ORDERED (OEQ, so NaN==NaN is false); the two are complements again only once Fne is UNORDERED (UNE). (2) OUT-OF-RANGE SHIFT DISTANCE LEFT TO THE HARDWARE: a Verilog-A integer is 32 bits so IEEE 1364 requires a distance >= 32 to give 0 (the right operand is unsigned, so a negative distance is huge and also gives 0), but openvaf emitted a bare shl/lshr/ashr and AArch64 masks the distance to 5 bits -- `1 << n` with n=32 gave 1 (i.e. 1«0) and `(-1) >> n` gave -1. Only the RUNTIME form was affected (a literal distance is poison, fixed in E-334), so the two spellings disagreed with each other as well as with the language. Now range-checked in codegen: « and » select 0 when the unsigned distance is >= 32, »> clamps to 31 which yields exactly the all-sign-bits result; ordinary shifts untouched. (3) THE SIMPLIFIER APPLIED ALGEBRA IEEE DOUBLES DO NOT OBEY: `Arithmetic for f64` set DIV_EXACT and HAS_SQRT true, enabling rewrites its own comment called "only fast math". With NODE VOLTAGES as operands these fired at run time -- `V(z)/V(z)` at z=0 gave 1, `sqrt(V(w))*sqrt(V(w))` and `exp(ln(V(w)))` at w=-4 gave -4, where IEEE requires NaN; `x-x`, `x+(-x)`, `x*0` and `x/-x` are unsound for the same reason. Each is exactly how a model guards a domain, so folding them replaced a deliberate NaN with a plausible wrong number. Gated on a new EXACT_ALGEBRA constant: true for i32 which really does obey algebra, false for f64, naming the property at each site instead of implying it from the type. Inverse-function cancellation tightened the same way -- the principal-value cases were already excluded, the DOMAIN/OVERFLOW ones were not (exp(ln x) for x<0, ln(exp x) and sinh/asinh on overflow, cosh(acosh x) for x<1, tanh(atanh x) for

E-336 *ngspice-46/src/osdi/osdiregistry.c, ngspice-46/src/osdi/osdiinit.c, openvaf/osdi/src/metadata.rs*

Three defects in how a compiled model is DESCRIBED to, and BOUND by, the simulator -- wrong values and a wrong array size, no crash. (1) AN INSTANCE PARAMETER NAMED `M` WAS CONSUMED AS THE MULTIPLIER: with `(* type="instance" *) parameter real M = 2.0` and `I <+ V*M`, an instance line `M=7` produced i(va)=14 = 7 x (1V x 2) -- applied as the DEVICE MULTIPLIER while the model's own M kept its default. ngspice decided whether to synthesize its `m` alias for $mfactor with a case-SENSITIVE strcmp(name,"m"), so a model declaring `M` never suppressed it, while that same M was lowercased to `m` on registration; both became `m` and the alias won. The inconsistency was visible in the same loop -- the dtemp/dt/temp tests two lines below already used strcasecmp. Now `m` does too and M=7 reaches the model's own parameter (i(va)=7). (2) PARAMETERS DIFFERING ONLY IN CASE SILENTLY LOST A VALUE: Verilog-A is case-SENSITIVE so `GAIN` and `gain` are distinct, SPICE is not, so both fold to one keyword and one loses -- `GAIN=3 gain=7` yielded 7.001, i.e. both routed to one parameter (last wins) while the other kept its default 1000. This CANNOT be resolved in the loader (a netlist is lowercased

when parsed, so the names are indistinguishable by the time a value arrives) but it must not be SILENT: it now warns, naming the module and parameter, so the author learns the names are unreachable instead of debugging a wrong answer. Binding behaviour deliberately unchanged; only the silence is fixed. (3) num_resistive_jacobian_entries EXCEEDED THE ENTRY LIST: 8 against num_jacobian_entries=7 -- a count larger than the array it describes, which a consumer trusting it would use to walk past the end. num_jacobian_entries and the entry list both come from the CURRENT dae_system.jacobian, but the resistive/reactive counts came from a value cached by count_jacobian_entries() while the DAE system was still being built, and the jacobian can lose entries afterwards (node collapsing), leaving the cached number stale. Both counts are now derived from the same map that produces the entry list -- consistent by construction rather than by timing. VERIFIED: an independent OSDI descriptor reader reports 0 violations on the two models that previously showed 2 and 1, and still 0 on ordinary models, so the checker is specific rather than blanket-passing. Verify (examples/osdiparam): 3 checks.

E-337 *openvaf/mir_opt/src/simplify.rs*

E-335 went too far: removing **`x * 0 -> 0`** changed HiSIM2's DC drain current by 10x, and the 92-model corpus campaign caught it. E-335 removed the IEEE-unsound algebraic rewrites; `x*0 -> 0` belongs to that family (`inf*0` and `NaN*0` are NaN, not 0) and went with them. Diffing the campaign against the pre-E-335 toolchain showed ONE of 92 device-modules had moved: hisim2_va, Id -1.30e-04 -> -1.33e-05 at Vg=0.7/Vd=1.0. Both Id-Vg curves were smooth and monotonic, so nothing looked broken -- which is why a DIFFERENTIAL was needed rather than an eyeball. ISOLATION: reverting the E-335 gates one at a time identified `x*0` as the sole cause -- restoring only that gate restored the value exactly, restoring any other did not. THAT IS ITSELF THE DIAGNOSIS: `x*0` is EXACT for every finite x, so removing the fold could only change a result if the operand were inf or NaN. HiSIM2 produces a non-finite intermediate at that bias which the fold had been silently absorbing. WHAT ACTUALLY TRIGGERS IT (two reproducer attempts missed this): one operand must be the interned CONSTANT zero and the other a RUNTIME value; if BOTH are constant, const_eval folds `0*inf` to NaN first (correctly) and the rewrite never applies; and a zero-valued PARAMETER is a runtime value, so `flag * term` with a zero-valued parameter flag does not trigger it either. The guarded case is a constant-zero coefficient multiplying a runtime non-finite term. DECISION: the fold stays, gated separately from EXACT_ALGEBRA and documented at the site. There is no evidence the un-folded answer is physically correct -- only that it differs -- and changing a production model's DC current by 10x for IEEE purity, on a fold whose unsoundness requires a non-finite operand the model was never expected to produce, is not a trade worth making. If that non-finite intermediate is itself a model defect, that is a finding about the model, not a reason for the compiler to change its answer. The rewrites that produced the REPORTED wrong answers (x/x -> 1, sqrt(x)*sqrt(x) -> x, exp(ln x) -> x, and the domain/overflow inverse cancellations) remain removed. VERIFIED: 92/92 corpus device-modules OK, worst KCL residual 2.3e-13 A; against the pre-E-335 toolchain NO device-module has a different drain current, and the only three differing rows differ solely in KCL residual at 1e-16/1e-17 -- orders of magnitude below the worst case and pure floating-point noise from the folds legitimately removed. All E-335 fixes survive. Verify (examples/vafmulzero): 2 checks.

E-340 *openvaf/hir/src/elaborate.rs*

Compiling the same source twice produced two different MIRs -- lrm_p150_1.va gave 2 distinct and lrm_p209_1.va gave 8, run to run on the SAME binary and input. Known and dismissed TWICE with the wrong cause attached; both earlier explanations were tested here and DISPROVED -- ”multi-module dump ORDER varies” (it was the timing line) and ”parallel module compilation” (RAYON_NUM_THREADS=1 still gives 2 hashes), as was a third guess, ”hash-container iteration of implicit nets” (implicit_nets is only get/insert, never iterated). ACTUAL ROOT CAUSE: diffing two UNOPTIMISED MIR variants reduced it to four lines, the first being the cause and the rest consequences -- `Spur(27)->'aa0'` vs `'aa2'` (interner ids), then swapped node numbering and SSA value numbering. Both names are implicit nets introduced

by ONE instance, `comparator C1(.cout(aa0), .inp(in), .inm(aa2))`. E-41 declares an undeclared connection actual implicitly, emitting the declaration the first time it meets each name while walking `for (port_name, binding) in port_raw.iter_mut()` -- and `port_raw` is a HashMap<Name, PortBinding>. Rust seeds its hashers RANDOMLY PER PROCESS, so the walk order varied every run; whichever implicit net was met first was declared first, interned first, and took the lower Spur. It takes TWO implicit nets on ONE instance to show -- with one there is nothing to reorder, which is why nearly the whole corpus is unaffected. RAYON_NUM_THREADS=1 does not help because the randomness is in the hasher SEED, not thread scheduling, which is exactly what made the parallelism theory look plausible and kept it alive. FIX: walk the bindings in the TARGET's declared port order -- deterministic, and the order a reader expects -- with the port name as a total tie-break. VERIFIED: the reproducer and both corpus models now yield ONE MIR over 12 compilations each (lrm_p209_1 went 8 -> 1); corpus-wide 488 models, 486 identical, 2 changed, NONDETERMINISTIC 0, the two changed being exactly the affected models now producing one canonical MIR instead of a random pick; and NOT a behaviour change -- the sigma-delta simulates byte-identically before and after (417 transient rows, same hash, four compilations per binary). WHY IT MATTERED: never a miscompile (the permutation is consistent, output unchanged), but builds were not bit-reproducible and it DEFEATED MIR-diff output-preservation checking -- a change under test could not be distinguished from the compiler disagreeing with itself. VA_TEST/mir_oracle.py --stable exists to tolerate exactly this; with the cause gone that flag is a safety net rather than a necessity. Verify (examples/vafdeterminism): 4 checks.

E-347 *OpenVAF-master-20260610/openvaf/mir_build/src/ssa.rs*

The SSA re-builder no longer mints an INVALID phi operand -- the deeper defect E-329 fixed the crash for and deliberately left open. The whole 496-file .va corpus now compiles under an ASSERTIONS build with zero panics. E-329 named the right FILE but the wrong PATH, and that mattered: the obvious fix -- repairing phis in FunctionBuilder::finalize(), where mir_build declares a function complete -- DID NOT WORK, because those phis do not exist yet. Instrumenting finalize() showed exactly ONE phi there, carrying no v0; a backtrace armed at phi creation showed the gravestone is minted by SSAVariableBuilder (the 'add values to an ALREADY FINISHED MIR function' builder) during topology linearisation, long after mir_build is done. That accounts for the symptom that had looked contradictory: validate() PASSES at sim_back/src/lib.rs:175 and FAILS at :179, on either side of Topology::new. Two other candidates were instrumented and CLEARED rather than eliminated by argument: the topology builder's own phi rebuild already defaults unmapped operands to F_ZERO (traced, never fires), and try_remove_phi_edge* in mir/src/dfg/phis.rs does write GRAVESTONE into an arg slot but only ever targeted inst11, not the failing inst27/inst30. An earlier ALIAS-TIMING hypothesis (value_def reports Invalid for Alias(_) too, so the operand might be an unresolved alias at finalize) was tested and REFUTED by the same instrumentation. Fix: in finish_predecessors_lookup, a GRAVESTONE operand now takes a LIVE SIBLING operand of the same phi. Why a sibling and not a synthesised zero -- the operand sits on an edge out of a block unreachable from the entry, so the edge CANNOT EXECUTE and the value is never read, making any sibling equally correct; and this builder is TYPE-AGNOSTIC (it holds no Type for the place), so a sibling is the only TYPE-CORRECT value available. F_ZERO would be wrong for an integer or boolean place. A phi whose operands are ALL gravestones is left alone. Fixing it here covers BOTH callers, which is what makes it the root fix rather than a patch at one call site. Output preservation: --dump-mir oracle over the corpus = 496 files, 495 IDENTICAL, 1 CHANGED, 0 nondeterministic -- and the one change IS the E-329 reproducer, whose diff is VALUE RENUMBERING ONLY (normalising v\d+ makes the dumps byte-identical; value count 131 -> 130, exactly what reusing an existing sibling instead of leaving a placeholder produces). It still computes E-329's own criterion exactly: ssn == ssn_ref at i=-5.0e-4, v=0.5. The three example models were each verified to TRIP the assertion on a PRE-FIX assertions build and to be clean after -- proven triggers, not decoration. A fourth candidate using a `for` loop did NOT trip and was DROPPED rather than shipped as filler. Regression 279/279.

Verify (examples/ssavalid): 3 checks (the definitive check needs an assertions build, and the doc records the command).

E-352 *openvaf/sim_back/src/dae.rs, dae/builder.rs, openvaf/osdi/header/osdi_0_4.h, openvaf/osdi/src/{lib,metadata,inst_data,load,eval}.rs, metadata/osdi_0_4.rs*

SUPERSEDED BY E-359 (the capability stands and is still verified by the same oracles; the compiler-emitted-tensor IMPLEMENTATION below was replaced by numerical differentiation of the analytic jacobian in ngspice, removing the compile-time regression, the 30MB objects, the runtime penalty and the ABI bump). **`.disto`** now includes a Verilog-A model's nonlinearities; previously it printed a warning and left them out. Distortion is a VOLTERRA analysis: it needs each device's Taylor expansion of I(v) to THIRD order, not the operating-point linearisation the Jacobian provides. Built-in devices hand-code those coefficients (diodset.c computes id_x2/id_x3), which is why only FOUR of ~58 built-ins implement DEVdisto at all -- diode, BJT, BSIM1, MES -- and the OSDI ABI stopped at first derivatives, so cktdisto had nothing to ask an OSDI device for. KEY ENABLER, verified before anything was built: OpenVAF's autodiff is ALREADY arbitrary-order (mir_autodiff's DerivativeIntern chains through `previous_order`, with a comment anticipating 8th order), and nested ddx(ddx(ddx(...))) reproduces closed form to every printed digit. So the compiler work was not implementing higher-order AD but ASKING for it: build_taylor_tensors re-runs auto_diff over its own first-order results for the second, and again for the third. NO 3-VARIABLE CEILING: ngspice's framework caps at three controlling variables (Dderivs holds derivatives "w.r.t 3 variables"; the DFx helpers take up to 27 scalar arguments with no fourth-variable form, DFn2F12 taking a struct because the list outgrew the calling convention). That cap belongs to those hard-coded helpers, not to the mathematics -- the contribution is a symmetric tensor contraction -- so osdidisto.c contracts directly over however many inputs a device has and never calls D1x/DFx. Proven with an ideal mixer I(out)=k*V(a, ref)*V(b,ref) whose ONLY nonlinearity is the mixed partial (both diagonal 2nd derivatives identically zero, so a dropped cross term gives exactly zero): matches closed form 0.5*k*A^2*R EXACTLY. TWO SILENT-ZERO BUGS found on the way, both indistinguishable from a linear model: (1) the tensors were OPTIMISED AWAY -- ensure_optbarriers protects residuals/noise/jacobian but the Taylor values feed nothing else in the MIR, so the optimiser deleted them and every coefficient arrived as zero (they also take the mfactor scaling, since m parallel devices inject m times the distortion current); (2) NOTHING STORED THEM -- eval's jacobian/residual stores are gated on CALC_ flags that .disto never sets. Plus one wrong-by-a-constant bug only a reference implementation could catch: D1nF12 returns 0.5*S2vF12(...) and S2vF12 ALREADY sums both index orderings, so without that 0.5 the mixed-tone products came out exactly 2x -- and because IM3 consumes the f1-f2 kernel its error was a non-obvious 2.246x. KNOWN LIMITATION AT THE TIME, SINCE LIFTED BY E-353: a model whose contribution goes through $limit emitted NO tensors (the residual depends on the limited value, not the raw voltage read; build_jacobian handled that via intern.lim_state and the tensor pass did not). Limiting is standard in production diode/BJT/MOS models, so this was a real restriction; E-353 folds the limited values into the derivative chain and removes it. What remains unreachable is a nonlinearity in a GROUND-REFERENCED probe (the tensors are indexed by model input and a bare V(a) is not one, having no hi/lo pair). Registering DEVdisto removed cktdisto.c's blanket warning, which would have turned such a model into a SILENT ZERO -- worse than before -- so OSDIdisto warns for itself. VERIFIED against four oracles, two involving no simulator at all: HD2/HD3 vs a closed-form polynomial (0.0 and 1.2e-16); the A^2/A^3 scaling laws (4.0000, 8.0000); f1+f2, f1-f2 and IM3 vs the built-in diode (1.87e-06, the bound the 0.24 ppm $vt difference justifies); IM3 vs transient+FFT in a DIFFERENT DOMAIN, converging 5.7% -> 1.4% -> 0.21% as the drive halves, exactly the A^2 signature of the 5th-order content a 3rd-order truncation excludes; and the multi-variable cross term (0.0). TWO HARNESS TRAPS recorded because both would have produced a confident wrong answer: the first IM3 transient was measuring SPECTRAL LEAKAGE from the fundamentals rather than IM3 (exposed by the A^3 law giving 2.93 instead of 8, not by the value looking wrong); and a first parameter choice made HD3 IDENTICALLY ZERO (the two contributions cancel exactly when c3 = 2c2^2/Ytot, and 2*(1e-4)^2/2e-3 is precisely the 1e-5

chosen) -- a zero that would have passed a naive check for the wrong reason, the same trap as the old campaign's "HD2 ~ 0 on a LINEAR network". The suite now refuses to score a both-zero comparison. Regression 284/284. The example is a proven trigger: on the pre-fix binaries 5 of its 6 checks fail, and the one that passes is the one that should. Verify (examples/osdidisto): 6 checks.

E-353 *openvaf/sim_back/src/dae/builder.rs, openvaf/osdi/src/load.rs, examples/limitdisto_examples/**

SUPERSEDED BY E-359 ($limit models still work -- the suite below passes unchanged -- but the chain-fold no longer exists; differencing the real jacobian makes the problem unable to arise). **.disto** now works for Verilog-A models that use $limit -- i.e. every production compact model. E-352 gave OSDI devices distortion analysis, but only for models reading their controlling voltages DIRECTLY; a model passing one through $limit contributed NOTHING, and limiting is how every production diode/BJT/MOS model converges, so the feature did not reach the models people actually run. ROOT CAUSE: $limit hands the model a LIMITED copy of a branch voltage and the residual is written in terms of that copy, so differentiating the residual by the RAW voltage read yields zero -- nothing in the expression depends on it. The model is not linear, but every derivative the tensor pass asked for was. This was never a problem for AC or transient: build_jacobian has always folded the limited values back in via intern.lim_state; E-352's build_taylor_tensors simply did not perform the same fold. FIX: each model input now maps to a CHAIN of autodiff unknowns rather than a single one -- new taylor_unknown_chain returns the limited values from intern.lim_state.raw (each with the sign it contributes) followed by the input's own unknown. The 2nd-order pass sums over every PAIR drawn from the two chains and the 3rd-order pass over every TRIPLE, each term carrying the XOR of its entries' signs; an entry is negated when the model limits a REVERSED branch, $limit(V(ref,a),...). VERIFICATION uses an exact expectation with no appeal to another simulator: $limit is a convergence aid, so at convergence the limited value equals the actual one and a model written with it is THE SAME DEVICE as the same model written without it -- every distortion product must agree. Both spellings are generated from ONE body string, which is what guarantees they differ only by $limit and cannot drift apart. Five shapes cover the chain combinatorics (where this can go wrong -- a model limiting a single branch never puts more than one entry on either side of a pair): two independently limited diagonals; a cross term with BOTH inputs limited (2x2 pairs); a cross term with ONE input limited (asymmetric chains, 2x1); a THIRD-order cross term whose chain has 3 entries, so the triple loop genuinely runs; and a reversed-branch limit, the only spelling producing a NEGATED chain entry. All 13 live comparisons agree, worst 7.1e-10 -- convergence noise, since the two spellings settle 8.3e-11 apart (limiting changes the Newton path, not the solution) and d2/dv2 of exp(v/vt) amplifies that by 1/vt^2. THREE WAYS THIS COULD HAVE PASSED WHILE WRONG, each caught and closed because each produces a confident green: (1) BOTH-ZERO comparisons -- 'no distortion' is precisely the pre-fix behaviour, so scoring 0 == 0 as agreement would certify the bug; the suite scores a both-zero product as a FAILURE except where the mathematics requires zero (a bilinear term k*v1*v2 has no third derivative, so its IM3 is correctly zero and is asserted as such). (2) COMPARING MAGNITUDES -- a dropped sign on a negated chain entry flips the result and leaves

E-359 *openvaf/sim_back/src/dae.rs, dae/builder.rs, openvaf/osdi/header/osdi_0_4.h, openvaf/osdi/src/{lib,metadata,inst_data,load,eval}.rs, metadata/osdi_0_4.rs, openvaf/test_data/{dae,init}/*.snap*

.disto for Verilog-A rebuilt on NUMERICAL differentiation of the analytic jacobian; the compiler-side tensor machinery is DELETED. E-352 obtained the 2nd/3rd-order Taylor coefficients by having the compiler emit a symbolic closed form. The capability was right but the cost was out of all proportion to what .disto does: on ASMHEMT 707k intermediate derivatives -> 1.4M MIR instructions -> a 30.2MB .osdi and a 126s compile against a 3.6s baseline (35x; hisimhv 20x), plus -- because the tensors were computed inside eval() -- a 3.3-3.9x RUNTIME penalty on every DC sweep, transient, AC and noise run, and an OSDI 0.8->0.9 ABI bump that made every already-compiled model unusable without a rebuild. ROOT MISTAKE: .disto needs the

coefficients at ONE point, once per instance, once per analysis; E-352 computed a closed form valid at EVERY point in order to evaluate it once. WHAT REPLACES IT: the model already publishes its FIRST derivatives analytically -- that is the jacobian every analysis loads -- so the higher ones follow by differencing THAT at the operating point (d2 R_r/dv_p dv_q = d/dv_q[J_rp]; d3 = d2/dv_q dv_s[J_rp]). Differencing an already-analytic first derivative rather than the residual is what keeps it accurate. Two properties make it cheap: the tensors are evaluated at the DC OPERATING POINT so they are frequency-INDEPENDENT (built once at D_SETUP, reused by every mode and every frequency), and they are built in NODE coordinates where an instance has 5-20 nodes rather than the 52 'model inputs' ASMHEMT reports. Entries keep the exact (row, col...) shape the analytic path produced, so the whole verified Volterra contraction is reused UNCHANGED -- only the tensor source and the kernel differ. NODE COORDINATES ALSO LIFT A REAL LIMITATION: E-352 indexed by model input (a branch voltage with a hi/lo pair), so a nonlinearity in a GROUND-REFERENCED probe V(a) had no pair, was never recorded, and the device reported 'contributes no distortion' and returned zero; it now works and is pinned to a closed form (-4.16666665e-02 vs exact -4.16666667e-02). E-353's $limit chain-fold disappears for the same reason -- differencing the real jacobian sees what the simulator sees, so there is no chain to fold, and E-353's seven-shape suite (written specifically to break that logic) passes unchanged against an implementation containing none of it. ACCURACY, measured against EXACT CLOSED FORMS rather than against the old implementation: cubic 4.0e-09/5.4e-09, exponential diode 2e-10/~1e-10, internal-node 3.6e-12/1.2e-08, ground-referenced 4e-09, MEXTRAM vs the analytic implementation 5e-09/1e-07 -- all below the 1.9e-06 floor the built-in-diode oracle already sits at (a 0.24 ppm $vt difference). THE STEP SIZE IS THE WHOLE GAME: truncation is O(h^2) and roundoff O(eps/h) at 2nd order but O(eps/h^2) at 3rd, so one step cannot serve both (a single 1e-3*V step put the result 8.5e-5 out); and the right yardstick is not the operating voltage but the CURVATURE scale (v_t = 26mV for a diode, volts for a polynomial), so a voltage-relative step left a cubic biased at 0V 11% wrong. The 2nd-order pass measures

E-363 *openvaf/mir_opt/src/simplify_cfg.rs, openvaf/hir/src/elaborate.rs*

Two compiler CRASHES on legal Verilog-A, found by a fuzzer that COMPOSES features each developed in isolation (mutation fuzzing cannot reach them -- mutants die at the parser). (1) `simplify_unconditional_jmp_term(src, dst)` merges `src` into `dst` -- retarget every predecessor, then delete `src` -- with no guard for `src == dst`. A block whose terminator jumps to ITSELF (`block2: jmp block2`, which is what a `case` inside a `do-while` folds to) made the retarget a no-op, and the block was deleted anyway while live terminators still named it; `mir_llvm::Builder::new` allocates LLVM blocks only for blocks IN the layout, so codegen unwrapped `None` (builder.rs: 655/656/690). The CFG layer already called this invalid -- `ControlFlowGraph::replace` opens with `debug_assert_ne!(old, new)`, exactly what fires on an assertions build. Now declines the self-merge: a self-loop is a legitimate CFG shape and must reach codegen. (2) A module has THREE array collections -- `buses`, `var_arrays` (E-4) and `param_arrays` (E-14) -- and instance flattening renamed only the first two, so every instance re-declared `cf[0]`, `cf[1]`, ... under the same names: a module with an array parameter could not be instantiated twice, and two DIFFERENT modules sharing an array-parameter name collided too. Now chains `param_arrays` into the rename. Equivalence: all 380 corpus files produce BYTE-IDENTICAL optimized MIR (the `.osdi` itself is not comparable -- the same binary on the same input hashes differently each run). Left OPEN and documented: a provably non-terminating analog loop still fails, because its contributions are then unreachable and there is no correct object code for such a model; the fix is a diagnostic, not a substituted value.